



Professional Vue.js Development

copyright 2026, Chris Minnick
version 2.0, 2026

INTRODUCTION AND VUE QUICKSTART

Introduction

Objectives

- Who am I?
- Who are you?
- Daily Schedule
- Course Schedule and Syllabus

About the Instructor

Introductions

- What's your name?
- What do you do?
- JavaScript level (beginner, intermediate, advanced)?
- What do you want to know at the end of this course?
- What do you like to do for fun?
- Favorite food?

Daily Schedule

- 08:30 - 10:30
- 15 minute break
- 10:45 - 12:00
- 1 hour lunch break
- 1:00 - 2:00
- 15 minute break
- 2:15 - 3:15
- 15 minute break
- 3:30 - 4:30

The Big Picture

- Topics Covered in this Course
 - Professional Front-End Development
 - Modern JavaScript
 - Vue.js

What's New in JavaScript (aka ECMAScript)?

- ES2015 (aka ES6) introduced many new features, including:
 - Arrow functions
 - Classes
 - Block-scoped binding constructs (let and const)
 - Iterators
 - Modules
 - Promises
- Since 2015, a new version is released annually
 - ES2020: optional chaining (?.), nullish coalescing (??)
 - ES2021: String.replaceAll, logical assignment
 - ES2022: top-level await, class fields, Array.at
 - ES2024: Object.groupBy, Promise.withResolvers

What is Vue.js?

- A progressive, incrementally adoptable, reactive front-end JavaScript framework

What is Reactivity?

- The way an application changes the view in response to actions.

History of Vue.js

- Created by Evan You after working for Google using AngularJS
 - "I figured, what if I could just extract the part that I really liked about Angular and build something really lightweight."
- Vue was originally released in February 2014.
- Vue 3 shipped in September 2020 and became the default in 2022.
- Vue 2 reached end of life on 31 December 2023.

What makes Vue different?

- Incrementally adoptable
 - You can use it with just a script include
 - or, you can use it with modern tooling
- Combines many of the best features of React and Angular
 - Has more built-in features than React
 - Less complex than Angular
- Small! (~34kb gzipped for the runtime + reactivity)
 - Unused features are tree-shaken out of the bundle
- Not created by a giant corporation
- Governed by an independent team, funded by its users

Comparison of Framework File Sizes

Angular 2 + Rx	766K
Angular 2	566K
Ember 2.2.0	435K
Ember 1.13.8	486K
Angular 1.4.5	143K
React 0.14.5 + React DOM	133K
React 0.14.5 + React DOM + Redux	139K
React 16.2.0 + React DOM	97.5K
Vue 2.4.2	58.8K
Inferno 1.2.2	48K
Preact 7.2.0	16K

Vue vs React and Angular

- Vue and Angular have 2-way data binding
 - Detects changes to the data model and updates the view automatically.
 - Detects changes to the view and updates the data model automatically.
- React has 1-way data binding
 - Developers write functions or use a state management system to update the model when events happen.
 - When the component re-renders, the view is updated from state.
- Vue and Angular have built-in utility attributes (directives)
 - v-for, v-on, v-bind, v-model
- React is mostly just pure JavaScript
- All three now favour a function-based, composable component style
 - Vue: the Composition API; React: hooks; Angular: signals

Demo: Vue.js Hello World

```
<div id="app">
  {{ message }}
</div>
```

```
import { createApp } from 'vue'
```

```
createApp({
  data() {
    return { message: 'Hello Vue!' }
  }
}).mount('#app')
```

```
// Vue 2 wrote: new Vue({ el: "#app", data: { ... } })
```


What Makes Vue Virtual DOM Different?

- Can be rendered at run time or precompiled
- Tracks a component's dependencies automatically
 - Knows when a component needs to be re-rendered
 - Avoids re-rendering components that haven't changed.

Vue Versions

- 1.0 - improved template syntax, v-for, hot reloading, scoped CSS
- 2.0 - virtual DOM, JSX support, server-side rendering
- 3.0 - Composition API, Teleport, Fragments, Proxy reactivity
- 3.2 - <script setup>, v-memo, custom-element build
- 3.3 - generic components, better TypeScript for defineProps
- 3.4 - faster parser, stabilised defineModel
- 3.5 - reactive props destructure, lazy hydration, lower memory

What Vue 3 Changed

- Reactivity rebuilt on Proxy
 - Added and deleted properties are now tracked; Vue.set is gone
- The Composition API, and `<script setup>` on top of it
- Fragments: a component may have more than one root element
- Teleport: render part of a component elsewhere in the DOM
- Multiple app instances, each isolated — there is no global Vue
- First-class TypeScript support
- Removed: filters, `$on/$off/$once`, functional component syntax

Vue QuickStart

Objectives

- Scaffold a project with create-vue

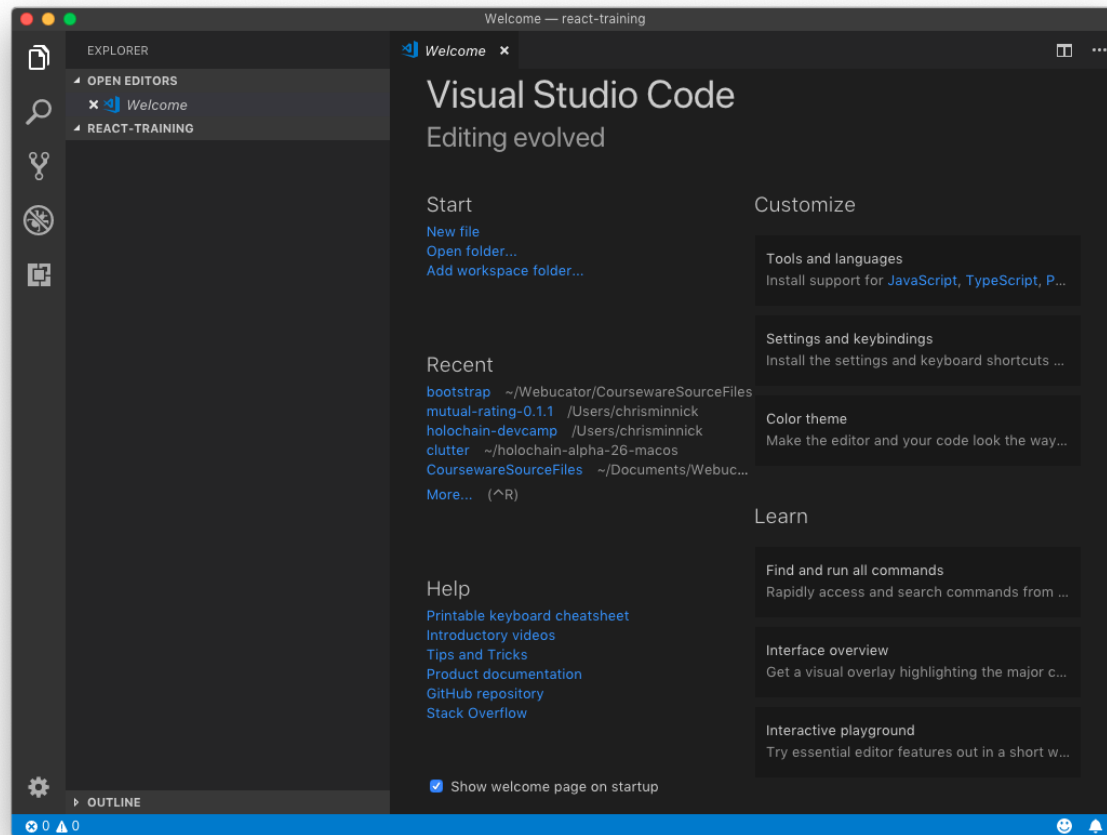
- Understand what Vite gives you

- Run, build and test a Vue app

Code Editors and IDEs

- Visual Studio Code
 - With the Vue - Official extension
- JetBrains WebStorm / IntelliJ IDEA
- Zed
- Sublime Text
- Neovim / Emacs
- Any editor with a Volar language-server client will do.

Lab 01: Installing Visual Studio Code



Vite

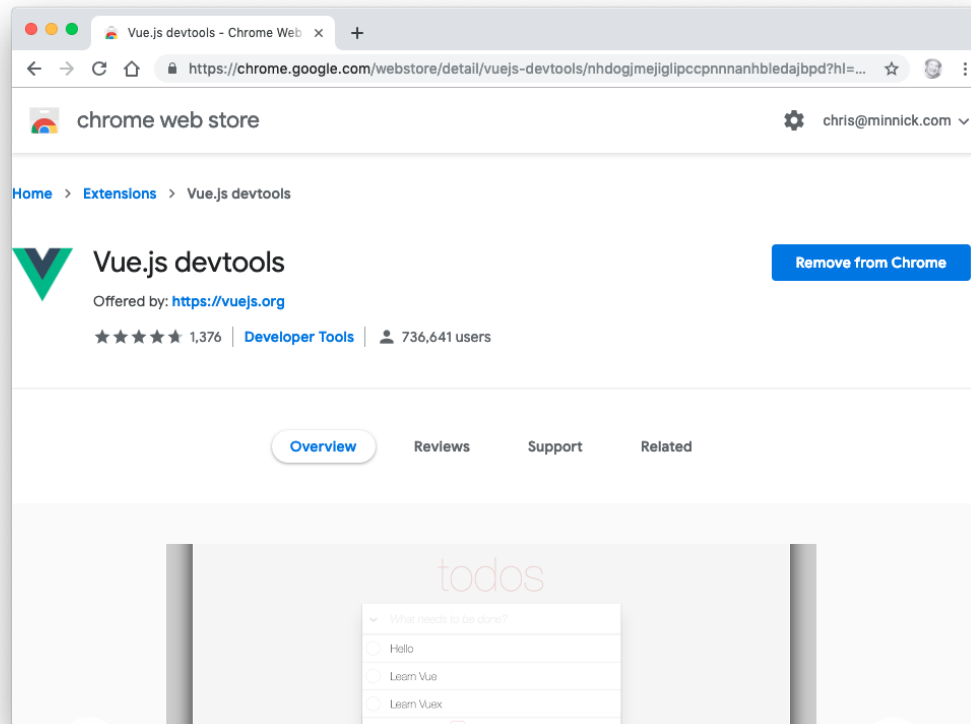
- The build tool behind the current Vue tooling
- Development: serves your source as native ES modules
 - No bundling step, so startup and reloads stay fast as the app grows
 - Hot module replacement that keeps component state
- Production: bundles and minifies with Rollup
- Framework-agnostic — Vue support comes from `@vitejs/plugin-vue`
- Vue CLI, which used webpack, reached end of life at the end of 2023

Using create-vue

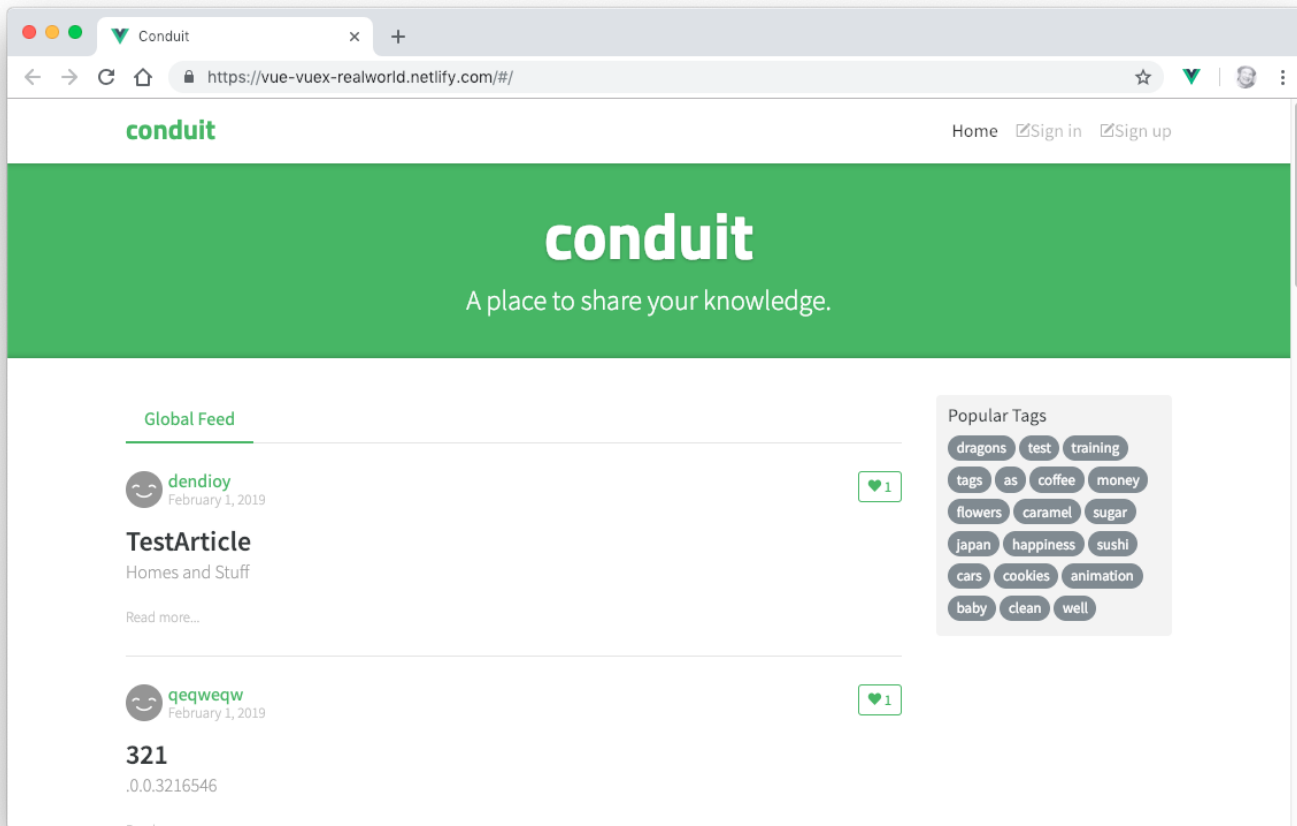
- Scaffold a project — nothing to install globally
 - `npm create vue@latest`
- Then, in the new directory:
 - `npm install`
 - `npm run dev` # start the dev server on :5173
 - `npm run build` # production build into dist/
 - `npm run preview` # serve the production build
- The prompts let you add Router, Pinia, Vitest, ESLint up front

Lab 02: Get Started with Vite

Installing and Using the Vue DevTools



Our Real-world Project: Conduit



Single-File Components

- `<script setup>` - the logic, using the Composition API
- `<template>` - the markup
- `<style scoped>` - CSS that applies to this component only
- One file, one component, named after the file.
- Compiled at build time, so the format costs nothing at runtime.

Lab 03: Your First Component

- Modern JS frameworks use modules to create reusable components.
- Components are composed into user interfaces.
- Vue's components are called "Single File Components"
 - They encapsulate Template, Script, and Style
- We write them with `<script setup>`, the Composition API syntax
 - Every top-level binding is available to the template
 - The import of a child component is its registration

Lab 04: Create More Components

- Practice building more components by creating the main sections of the Conduit demo app.

Lab 05: Testing Vue

- Write your first Vue tests with Vitest and Vue Test Utils.

Node.js

Objectives

- Install Node
- Test node and get started using npm

What is Node.js?

- JavaScript runtime
- Built on Google Chrome's V8 JavaScript engine
- Open-source
- Event-driven
- Non-blocking
 - Never waits
- Single-threaded
 - Can handle thousands of concurrent connections with minimal overhead
- No buffering
- Used on the server as well as for development automation

How is Node.js Different?

- JavaScript is single-threaded
- A Node application has one event loop, and gives you the benefit of multithreading with async operations
 - Can handle thousands of concurrent connections with minimal overhead
- No buffering
 - Node applications never buffer data.
 - Data is output in chunks.
- Non-blocking
 - Node doesn't wait for data to be returned from APIs.
 - While it's possible to write blocking code in Node.js, it's discouraged.

Event loop

- Problem: JavaScript is single-threaded.
- The event loop allows Node.js to perform non-blocking I/O operations.
 - Offloads operations to the system kernel when possible
- Node initializes the event loop on startup
- Loops through 6 phases
- Between each run of the loop, Node.js checks if it's still waiting for anything and shuts down if not.

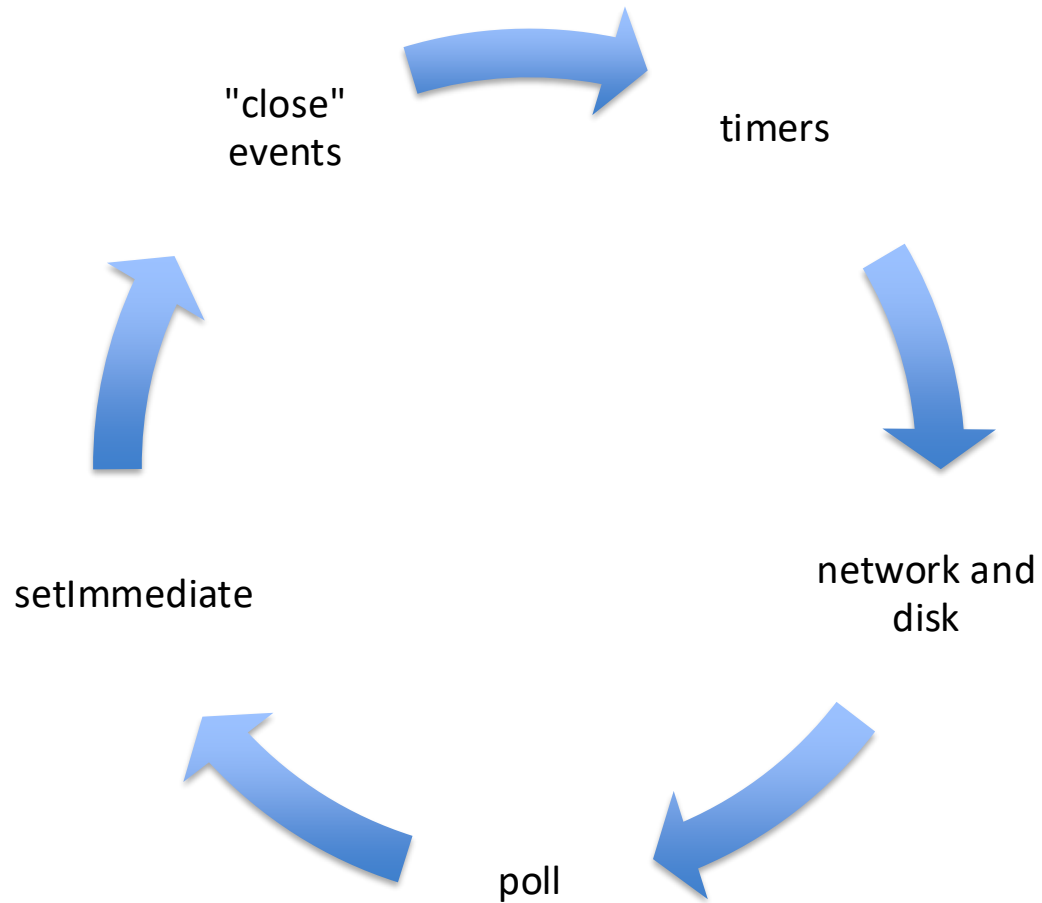
How is Node.js Different?

- JavaScript is single-threaded
- A Node application has one event loop, and gives you the benefit of multithreading with async operations
 - Can handle thousands of concurrent connections with minimal overhead
- No buffering
 - Node applications never buffer data.
 - Data is output in chunks.
- Non-blocking
 - Node doesn't wait for data to be returned from APIs.
 - While it's possible to write blocking code in Node.js, it's discouraged.

Event Loop Phases

1. timers - executes `setTimeout()` and `setInterval()` callbacks
2. I/O callbacks - executes most other callbacks
 - except close callbacks and `setImmediate()`
3. idle, prepare - internal use
4. poll - retrieve new I/O events
5. check - `setImmediate()` callbacks are executed
6. close callbacks - for example `socket.on('close', ...)`

Event Loop



EventEmitter

- Node is event driven.
- EventEmitter is a class that lets you listen for events and assign actions when those events occur.
- Very loose way of coupling different parts of your application together.

```
var EventEmitter = require("events").EventEmitter;
```

```
var ee = new EventEmitter();  
ee.on("someEvent", function () {  
    console.log("event has occurred");  
});
```

```
ee.emit("someEvent");
```

A Simple Node.js Example

```
const http = require('http');

const hostname = '127.0.0.1';
const port = 3000;

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello World\n');
});

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```


Modules Overview

- Node uses CommonJS for loading modules
- Original name was ServerJS
- Provides a way to import dependencies in JavaScript when used outside the browser (such as server side or in desktop applications)

`module.exports`

- Encapsulates code into a module

`require`

- Imports a module into JavaScript code

CommonJS Example

foo.js

```
var circle = require('./circle.js');  
console.log('Area of circle: ' + circle.area(4));
```

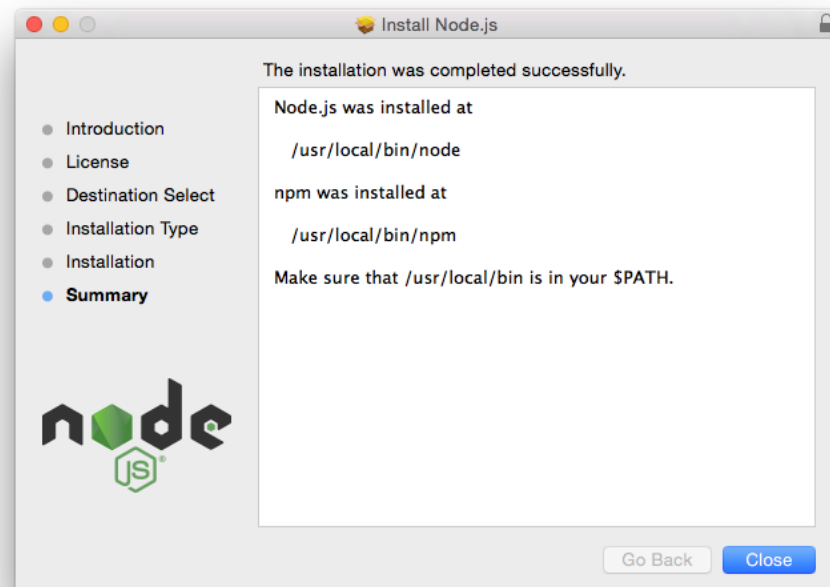
circle.js

```
exports.area = function(r) {  
  return Math.PI * r * r;  
}
```

Front-end Node

- Node has become an essential part of front end development.
- It's used by:
 - Package installers
 - Task runners
 - Testing frameworks
 - CSS compilers
 - JS Transpilers
 - Web servers
 - Web browsers (PhantomJS)
 - and more!

Lab 06 - Getting Started with Node.js



TOOLING

Why Have Tooling?

- Front-end Web Development is complex.
- Modern Web Tooling helps achieve four best practices of software development:
 - Version Control
 - Automation
 - Reproducible Build
 - Test-Driven Development

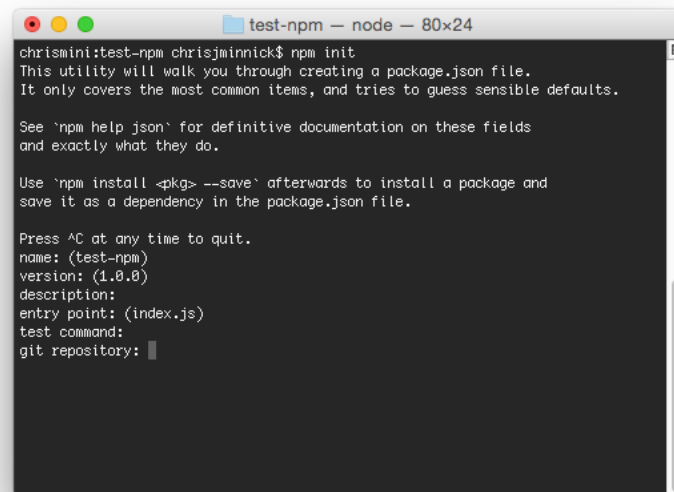
npm

- Installs, publishes, and manages node programs
- Is bundled and installed with Node
- Allows you to install Node.js applications from the npm registry
- Written in JavaScript

Other package managers

- npm ships with Node.js and is the default
- yarn — created by Facebook; Plug'n'Play install mode
- pnpm — hard-links a global store, so disk use stays flat
 - Strict by default: a package can only import what it declares
- bun — a runtime with a very fast built-in installer
- All four read package.json; they differ in the lockfile

Lab 07 – Initialize npm

A terminal window titled "test-npm -- node -- 80x24" showing the output of the "npm init" command. The window has a dark background and a light border. The text inside is white. The prompt is "chrismini:test-npm chrisjminnick\$". The output of "npm init" is as follows:

```
chrismini:test-npm chrisjminnick$ npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help json` for definitive documentation on these fields
and exactly what they do.

Use `npm install <pkg> --save` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
name: (test-npm)
version: (1.0.0)
description:
entry point: (index.js)
test command:
git repository: 
```

node_modules

- Two ways to install npm packages
 - Locally
 - Globally
- When you install packages locally, they're put into the **node_modules** directory in your current directory.
- You should always run npm install from the same directory as your **package.json** file, which should be at the root of your project.
- Run `npm update` to update local packages
- Run `npm outdated` to find out which packages are outdated.

package.json

- Manages locally installed npm packages
- Documents packages your project depends on
- Specifies the versions of each package your project can use
- Makes your build reproducible
- Package versions are specified using Semantic versioning (semver)
- Semver ranges:
 - ~ : patch release - 1.0.x
 - ^ : minor release - 1.x
 - * : major release - x

npm install

- Is used to download and install a package
 - (no flag): saves to dependencies — this is the default
 - --save-dev (-D): saves to devDependencies
 - --save-optional: saves to optionalDependencies
 - --save-exact (-E): pins the exact version, no range operator
 - --global (-g): installs to a shared location, not the project
- npm ci installs strictly from the lockfile — use it in CI

package.json

- Every npm package contains this file
- Holds metadata relevant to the project
 - Identifier Metadata
 - Name
 - Version
 - License
 - Description
 - Functional Metadata
 - Entry point
 - Repository
 - Scripts
 - Dependencies
 - Dev Dependencies

Lab 08: Making a Web Server

Demo / Code-along: Set up your own tooling

- How to create a Vue.js app with single-file components, starting from an empty directory.
- github.com/watzthisco/pro-vue/demos/Vue-Starter
- Three dependencies: vue, vite, @vitejs/plugin-vue

Static Code Analysis

Objectives

- Learn about lint tools

- Use ESLint

- Configure ESLint with a flat config

- Manual testing with a local dev server

Lint tools

- ESLint
 - The standard. Pluggable, and "agenda free" - it does not push a particular style
 - Flat config (eslint.config.js) since version 9
- Biome
 - Linter and formatter in one, written in Rust
- oxlint
 - A very fast linter, also Rust; often run alongside ESLint
- Prettier
 - A formatter, not a linter - it is about layout, not correctness
- JSLint and JSHint came first, and are now rarely used.

Configuring ESLint

- Two ways
 - Configuration comments
 - embed configuration info in JS files with comments
 - `/* eslint eqeqeq: "off", curly: "error" */`
 - Configuration files
 - `eslint.config.js` — the "flat config" format
- Flat config became the default in ESLint 9
 - It exports an array; each entry applies to the files it matches
 - `.eslintrc.*` and `.eslintignore` were removed in ESLint 10

ESLint: What Can Be Configured?

- `languageOptions`
 - `ecmaVersion`, `sourceType`, and the parser to use
 - `globals` — the predefined variables the code can rely on
- `files / ignores`
 - Which paths this configuration object applies to
- `plugins`
 - Imported objects now, not strings resolved by name
- `rules`
 - Enable rules at different levels

ESLint Rules

- 3 Levels
 - "off" or 0
 - Rule not applied.
 - "warn" or 1
 - Warn but don't exit.
 - "error" or 2
 - Error and exit.

Example rules

```
{
  "rules": {
    "eqeqeq": "off",
    "curly": "error",
    "quotes": ["error", "double"]
  }
}
```

Lab 09 - Automate Linting

- Install ESLint into the project
- Write an eslint.config.js flat config
- Create an npm script called "lint"
- Make the lint task a dependency of the default build task

Transpilers

- Not all browsers support the same JavaScript and CSS features.
- <https://caniuse.com>
- Transpilers convert modern code to a form older browsers accept.
- Vite does this for you at build time, guided by browserslist.
 - esbuild handles the syntax; the target defaults to browsers with native ES module support.

Browser compat-table

Feature name	Current browser	Traceur	Babel 6 + core-js 2	Babel 7 + core-js 2	Babel 7 + core-js 3	Closure 2019.02	Type-Script + core-js 2	Type-Script + core-js 3	es7-shim	IE 11	Edge 17	Edge 18	Edge 19 Preview	FF 60 ESR	FF 64	FF 65	FF 66
Proxy internal calls, Array.prototype.includes	Yes	No	No	No	No	No	No	No	No	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes
2017 features																	
• Object static methods	4/4	0/4	4/4	4/4	4/4	3/4	4/4	4/4	3/4	0/4	4/4	4/4	4/4	4/4	4/4	4/4	4/4
• String padding	2/2	0/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2	0/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2
• trailing commas in function syntax	2/2	0/2	2/2	2/2	2/2	2/2	2/2	2/2	0/2	0/2	2/2	2/2	2/2	2/2	2/2	2/2	2/2
• async functions	16/16	4/16	4/16	4/16	4/16	10/16	9/16	9/16	0/16	0/16	16/16	16/16	16/16	16/16	16/16	16/16	16/16
• shared memory and atomics	17/17	0/17	0/17	0/17	0/17	0/17	0/17	0/17	0/17	0/17	0/17	0/17	0/17	0/17	0/17	0/17	0/17
2017 misc																	
• Proxy "ownKeys" handler, duplicate keys for non-extensible targets (ES 2017 semantics) [22]	Yes	No	No	No	No	No	No	No	No	No	Yes	Yes	Yes	No	No	No	No
• RegExp "u" flag, case folding	Yes	No	No	No	No	No	No	No	No	No	No	No	No	Yes	Yes	Yes	Yes
• arguments.caller removed	Yes	No	No	No	No	No	No	No	No	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes
2017 annex b																	
• Object.prototype getter/setter methods	16/16	0/16	16/16	16/16	16/16	0/16	16/16	16/16	0/16	8/16	14/16	14/16	14/16	16/16	16/16	16/16	16/16
• Proxy internal calls, getter/setter methods	4/4	0/4	0/4	0/4	0/4	0/4	0/4	0/4	0/4	0/4	4/4	4/4	4/4	4/4	4/4	4/4	4/4
• assignments allowed in for-in head in non-strict mode	Yes	Yes	No	No	No	No	No	No	No	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes
2018 features																	
• object rest/spread properties	2/2	0/2	2/2	2/2	2/2	0/2	2/2	2/2	0/2	0/2	0/2	0/2	0/2	2/2	2/2	2/2	2/2
• Promise.prototype.finally	3/3	0/3	3/3	3/3	3/3	3/3	3/3	3/3	0/3	0/3	0/3	3/3	3/3	3/3	3/3	3/3	3/3
• s (dotAll) flag for regular expressions	Yes	?	Yes	Yes	Yes	No	?	?	No	No	No	No	No	No	No	No	No
• RegExp named capture groups	Yes	No	Yes	Yes	Yes	No	No	No	No	No	No	No	No	No	No	No	No
• RegExp Lookbehind Assertions	Yes	No	No	No	No	No	No	No	No	No	No	No	No	No	No	No	No
• RegExp Unicode Property Escapes	Yes	No	Yes	Yes	Yes	No	No	No	No	No	No	No	No	No	No	No	No

Babel and esbuild

- Babel is a transpiler.
 - Plugin-based, and the reference implementation for new syntax
- Vite uses esbuild instead for its dev-time transform
 - Written in Go, and roughly two orders of magnitude faster
- You rarely configure either one directly in a Vite project

Transpiling Example

ES2015

```
let myFunc = ()=>{  
  let myVar = {  
    myProp,  
  };  
};
```

ES5

```
"use strict";  
  
var myFunc = function  
myFunc() {  
  var myVar = {  
    myProp: myProp  
  };  
};
```

Bundlers

A bundler resolves your import graph and writes deployable files.

webpack — the incumbent; configured with loaders and plugins

Rollup — what Vite uses for production builds

esbuild — what Vite uses to pre-bundle dependencies

In development Vite does not bundle your source at all.

How Vite Works

- In development
 - Serves your source over HTTP as native ES modules
 - Transforms each file only when the browser asks for it
 - Pre-bundles dependencies once with esbuild, then caches them
- In production
 - Bundles with Rollup: tree-shaking, code-splitting, minification
 - Each lazily-imported route becomes its own chunk
- Plugins extend both pipelines — `@vitejs/plugin-vue` compiles SFCs

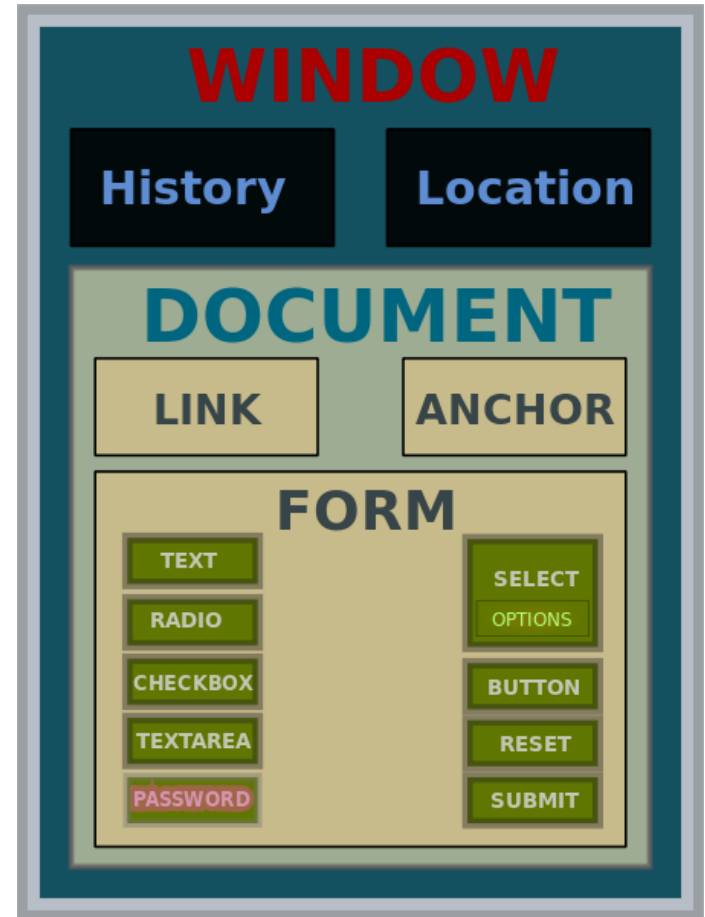
The Document Object Model

Objectives

- Understand how the DOM works
- Select DOM nodes
- Manipulate the DOM with JavaScript

What is the DOM?

- JavaScript API for HTML documents
- Represents elements as a tree structure
- Objects in the tree can be addressed and manipulated using methods.



JohnManuel [GFDL (<http://www.gnu.org/copyleft/fdl.html>) or CC BY-SA 3.0 (<http://creativecommons.org/licenses/by-sa/3.0/>)], via Wikimedia Commons

Understanding Nodes

- DOM interfaces that inherit from Node
 - Document
 - Element
 - CharacterData
 - Text
 - Comment
 - DocumentType
- Nodes inherit properties from EventTarget

EventTarget

- An interface implemented by objects that can receive events (aka event targets)
- Examples:
 - Element
 - Document
 - Window
- Many event targets support setting event handlers.

DOM Events

- Things that happen in the DOM

abort	focusout	resize
beforeinput	input	scroll
blur	keydown	select
click	keyup	unload
compositionend	load	wheel
compositionupdate	mousedown	
dblclick	mouseenter	
error	mouseleave	
focus	mousemove	
focusin	mouseout	
	mouseover	
	mouseup	

Other Events

- Many interfaces implement events or inherit events.
- Visit *<http://developer.mozilla.org/en-US/docs/Web/Events>*.

Element

- Interface for elements within a Document
- Inherits properties and methods from Node and EventTarget

- **Most common properties:**

`innerHTML`

`attributes`

`classList`

`id`

`tagName`

- **Most common methods**

`getElementById`

`addEventListener`

`querySelectorAll`

Manipulating HTML with the DOM

- You can get and set properties of HTML elements with JavaScript through the DOM

```
//starting HTML and DOM Element
```

```
<p id="favoriteMovie">The Matrix</p>
```

```
<script>
```

```
getElementById("favoriteMovie")
```

```
    .innerHTML = "The Godfather";
```

```
</script>
```

```
//updates DOM, which updates the browser
```

```
<p id="favoriteMovie">The Godfather</p>
```

Manipulating HTML with the DOM

```
<ol id="favoriteSongs">  
  <li class="song"></li>  
  <li class="song"></li>  
  <li class="song"></li>  
</ol>
```

```
<script>  
var mySongs=document  
  .querySelectorAll("#favoriteSongs .song");  
mySongs[0].innerHTML = "My New Favorite Song";  
</script>
```

Manipulating HTML with JQuery

```
<ol id="favoriteSongs">  
  <li class="song"></li>  
  <li class="song"></li>  
  <li class="song"></li>  
</ol>
```

```
<script>  
$("#favoriteSongs .song").first()  
  .html("My New Favorite Song");  
</script>
```

Manipulating HTML with React

```
<div id="favoriteSongs"></div>
```

```
function FavoriteSongs({ song }) {  
  return (  
    <ol>  
      <li className="song">{song}</li>  
    </ol>  
  );  
}
```

```
createRoot(document.getElementById('favoriteSongs'))  
  .render(<FavoriteSongs song="My New Favorite Song"  
/>);
```

Manipulating HTML With Vue.js

```
<div id="favoriteSongs">
  <ol>
    <li v-for="song in songs" :key="song.title" class="song">
      {{ song.title }}
    </li>
  </ol>
</div>
```

```
import { createApp } from 'vue'
```

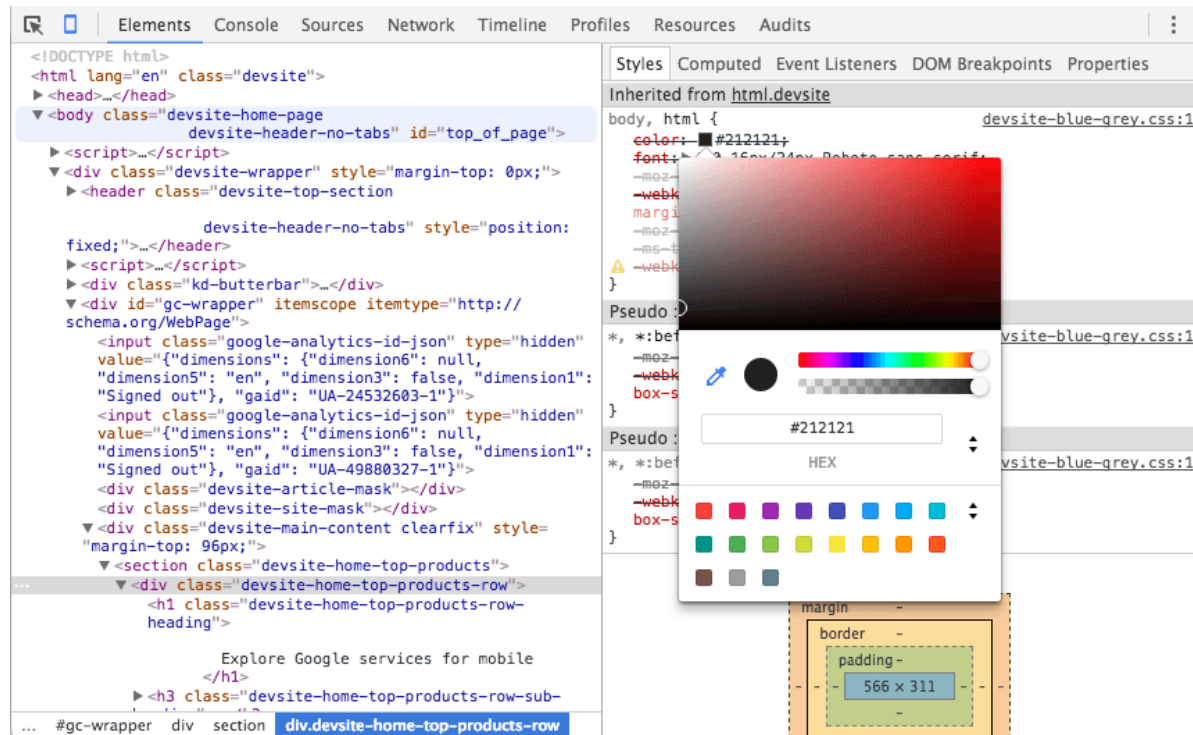
```
createApp({
  data() {
    return {
      songs: [
        { title: 'Walk This Way' },
        { title: 'Give It Away' },
        { title: 'My Way' }
      ]
    }
  }
}).mount('#favoriteSongs')
```

DEBUGGING JAVASCRIPT

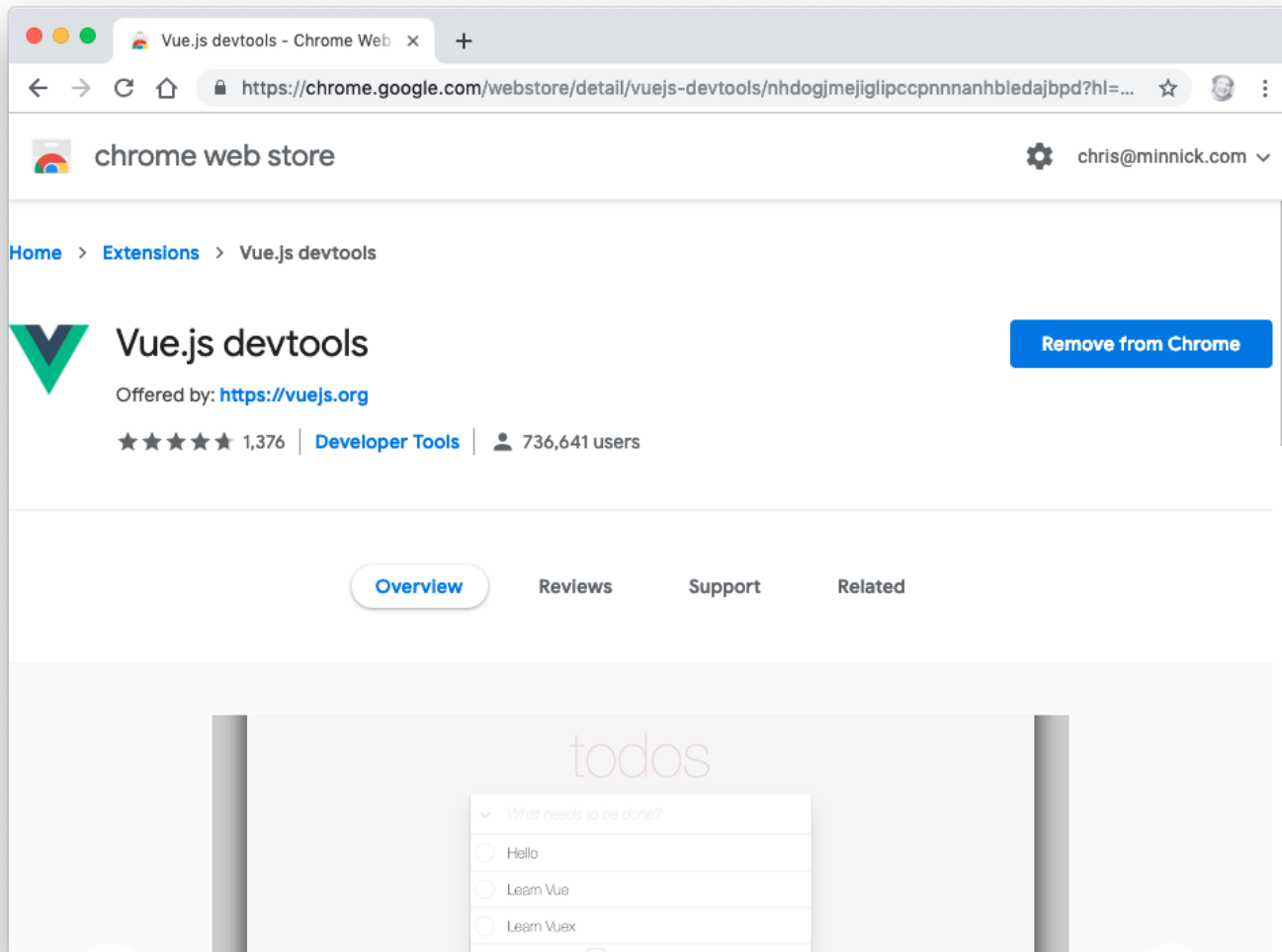
Browser Development Tools

- Using Chrome Developer Tools
- Using IE F12 Developer Tools

Installing and Using the Vue DevTools



Lab 10: Manual In-Browser Testing and Debugging



GETTING STARTED WITH VUE.JS

BASIC VUE.JS FEATURES

Options API vs. Composition API

- Options API
 - Code is grouped by option: data, computed, methods, watch
 - A single feature is spread across several options
 - Reuse means mixins
- Composition API
 - Code is grouped by feature
 - Reuse means composables - ordinary functions
 - Better type inference and IDE support
- Both are fully supported. This course uses the Composition API, which is what current Vue code is written with.

Creating and Mounting an App

```
import { createApp } from 'vue'
import App from './App.vue'

const app = createApp(App)    // an isolated application
instance

app.use(createPinia())        // plugins are installed
per app
app.use(router)
app.mount('#app')             // where to render

// There is no global Vue object, so two apps on the
// same page
// never share plugins, components, or configuration.
```

<script setup>

```
<script setup>
import { computed, ref } from 'vue'
• import ArticleList from './ArticleList.vue' // and registered

const count = ref(0) // state
const doubled = computed(() => count.value * 2)

function increment() { count.value++ } //
behaviour
</script>

// Every top-level binding is available to the
template.
// No export default, no components object, no `this`.
```


ref and computed

- `ref(value)` wraps any value in a reactive container
 - Read and write it as `.value` in script
 - Vue unwraps it for you in the template
- `computed(getter)` derives a cached, read-only value
 - Re-evaluates only when something it read has changed
- `reactive(object)` makes an object reactive without `.value`
 - Objects only, and it does not survive destructuring
- Reach for `ref` by default.

Compiler Macros

- Available only at the top level of <script setup>. They are not imported - the compiler handles them.
- `defineProps({ ... })` - declare the props
- `defineEmits(['change'])` - declare the events
- `defineModel()` - a writable v-model ref (3.4+)
- `defineExpose({ ... })` - expose members to a template ref
- `defineOptions({ ... })` - set component options such as name

App-level Configuration

`app.component(name, Component)` — register globally

`app.directive(name, definition)` — register a custom directive

`app.use(plugin)` — install a plugin

`app.provide(key, value)` — provide a value to every descendant

`app.config.errorHandler` — catch errors from any component

`app.mount(selector)` / `app.unmount()`

Vue 2 set all of these on the global Vue; Vue 3 scopes them to the app instance.

```
<script setup>
import { computed, ref } from 'vue'

const message = ref('Hello')           // state
const shouted = computed(() =>
message.value.toUpperCase())

function reset() { message.value = '' } //
behaviour
</script>
```

VUE TEMPLATE SYNTAX

Vue Templates

- Vue compiles templates to Virtual DOM render functions.
- Vue Templates are most often written using HTML, but can also be written using JSX or using JavaScript directly.

Using HTML in Templates

- Uses mustache syntax for text interpolation

```
<div>{{ customerName }}</div>
```

- To output HTML, use the v-html directive

```
{{ info }} //outputs as plain text
```

– `<div v-html="info"></div>` //output as HTML

Using JavaScript in Templates

- Vue supports using JavaScript expressions inside templates and in data bindings.

```
{{ counter++ }}
```

```
{{ loggedIn ? <LogOut /> : <LogIn /> }}
```

```
<div v-bind:id="'list-' + id"></div>
```

Vue.js Directives

- Attributes that take a JavaScript expression as their value.
- They have a prefix followed by a directive ID
- Example:

```
<div v-if="test">The test is true</div>
```


Directive Arguments

- Directives may take an argument
- : is used between directive ID and the argument.

```
<button v-on:click="doSomething">Click  
Me</button>
```

Directive Modifiers

- Some directives have modifiers that bind the directive in some particular way.
- These modifiers follow a . after the argument.

```
<form v-on:submit.prevent="doSomething">...</form>
```

.prevent will call event.preventDefault to prevent the default form action prior to calling the doSomething function.

Vue.js Default Directives

- Prefixed with v-
- v-bind
 - binds attributes to data
- v-if
 - conditional rendering
- v-for
 - loops over arrays
- v-on
 - event handling
- v-model
 - 2-way binding between inputs and app state

Custom Directives

- Components are the main way to reuse code.
- Still, there may be times when you need low-level DOM access on plain elements.

```
const vFocus = {  
  mounted: (el) => el.focus()  
}
```

```
<input v-focus>
```

```
// In <script setup>, a const named vName is usable as v-name.  
// The hook names now match the component lifecycle: what Vue 2  
// called "inserted" is "mounted".
```

Loops and Lists

- Use the v-for directive to render a list of items from an array or object

```
<ul>
  <li v-for="song in songs" :key="song.id">
    {{ song.title }}
  </li>
</ul>
```

```
const songs = ref([ { id: 1, title: 'Hotel California' } ])
```

Using key

- Always provide a key with v-for.

```
<li v-for="song in songs" :key="song.id">
  {{ song.title }}
</li>
```

- Providing a key helps Vue track each node's identity.
- The key goes on the element that carries v-for — including on a <template v-for>, which Vue 2 did not allow.
- Use a stable id, not the array index.

Using v-for

- Renders a list based on an array or object
- Often used with HTML lists

v-for with Arrays

```
<ul id="favorite-songs">
<li v-for="song in songs">
  {{song.title}}
</li>
</ul>

...
songs: [
  {title: 'Hotel California'},
  {title: 'April in Paris'}
]

...
Result:
Hotel California
April in Paris
```


v-for with Objects

- When used with an object, v-for iterates over the properties of the object.

```
<ul id="object-for-example">
  <li v-for="value in object">
    {{value}}
  </li>
</ul>
```

...

```
object: {
  firstName: 'Joe',
  lastName: 'Smith'
}
```

...

Result:

Joe

Doe

Conditional Rendering

- Use v-if / v-else-if / v-else

```
<h1 v-if="thing === true">It is so</h1>
```

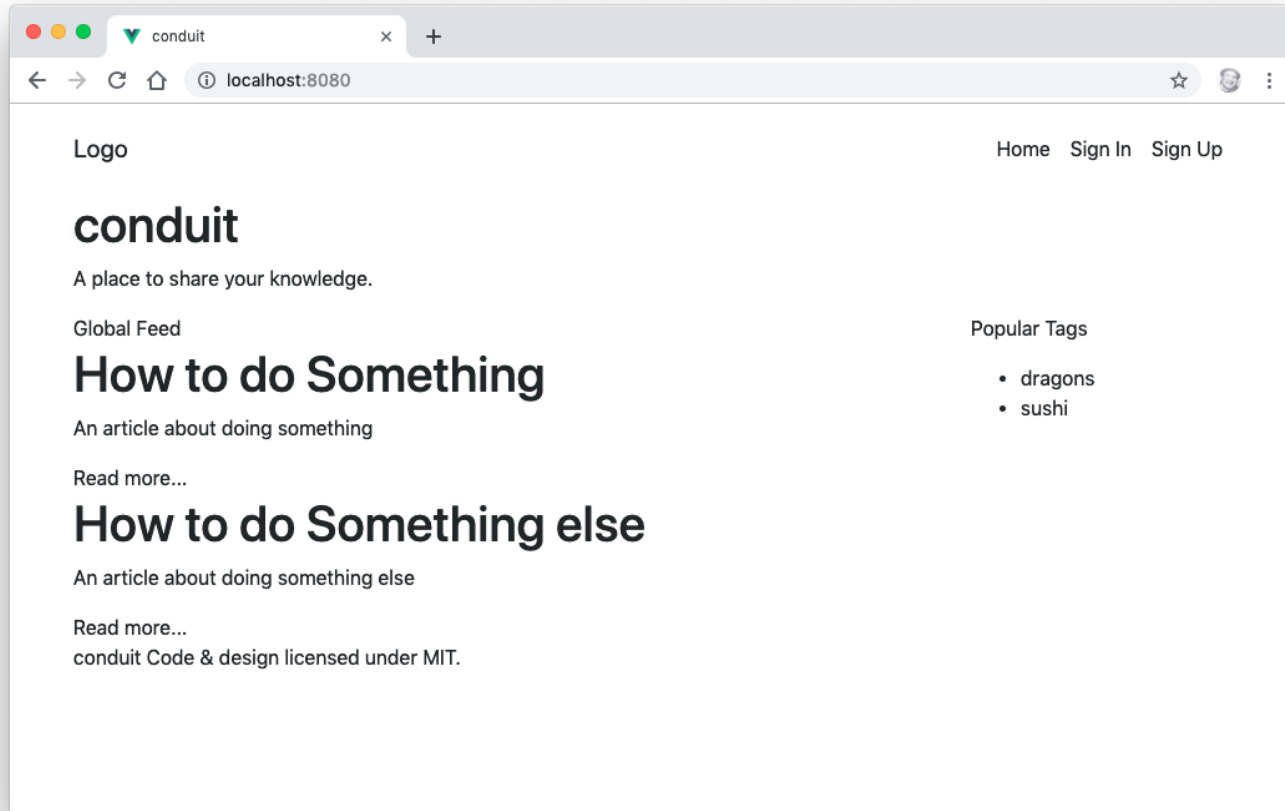
```
<h1 v-else-if="thing === undefined">We don't  
know</h1>
```

```
<h1 v-else>Is it not so</h1>
```

v-if vs. v-show

- v-show can be used for conditional rendering also
- The difference is that v-show only toggles the CSS display property.

Lab 11: Static Vue View



Binding HTML Classes

- `:class` can be used to dynamically toggle classes

```
<div :class="{ active: isActive }"></div>
```

- pass an array to `:class` to apply a list of classes

```
<div :class="[baseClass, { active: isActive  
}]"></div>
```

- Vue merges these with any static class attribute on the element.

Adding Styles Conditionally

- You can bind a computed property or a ref to the class attribute to change styles based on conditions.

```
<div :class="{ active: isActive },  
errorClass"></div>
```

```
const errorClass = computed(() =>  
(hasError.value ? 'text-danger' : ''))
```

Binding Styles

- Inline object syntax for :style

```
<div :style="{ color: activeColor, fontSize: fontSize + 'px' }">  
</div>
```

- Bind to a style object to keep the template cleaner

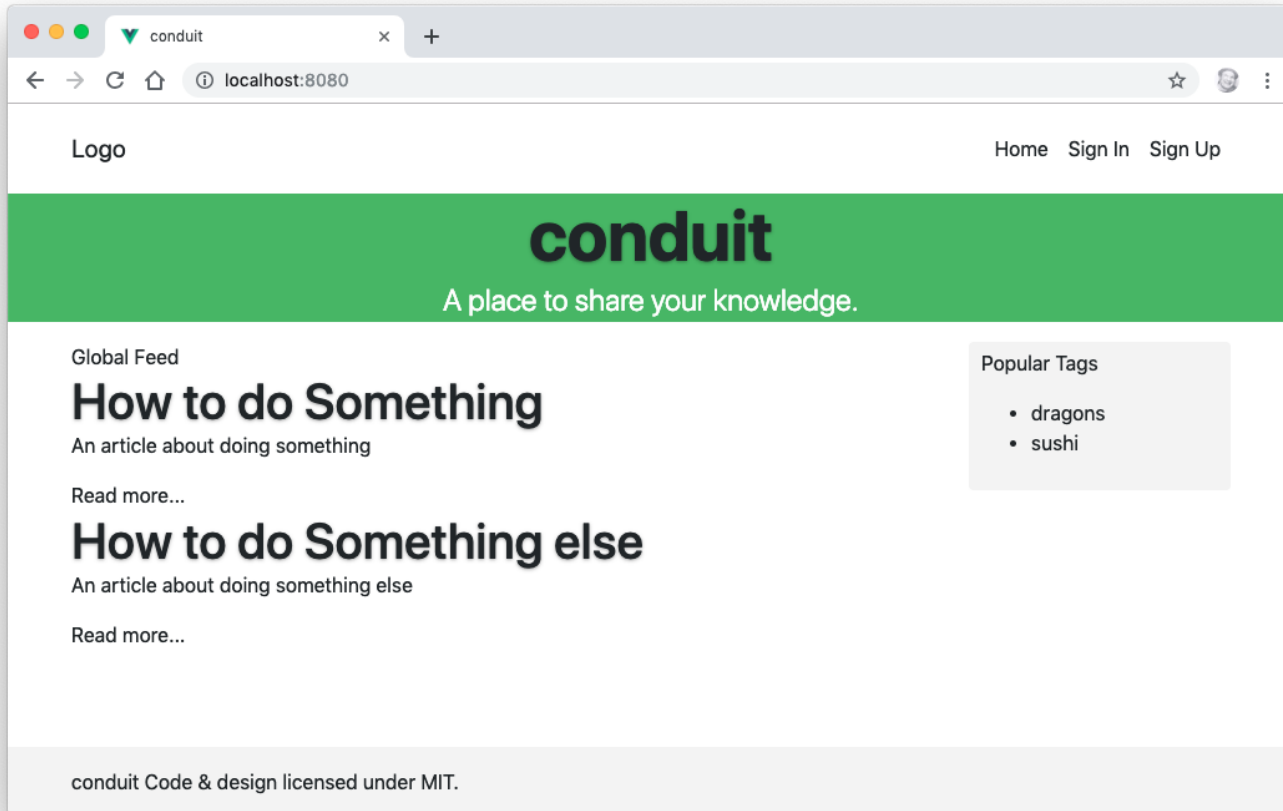
```
<div :style="styleObject"></div>
```

```
const styleObject = reactive({  
  color: 'red',  
  fontSize: '13px'  
})
```

DEMO

OUTPUTTING RAW HTML

Lab 12: Styling Vue Components



```
import { computed, ref } from 'vue'

const firstName = ref('Ada')
const lastName = ref('Lovelace')

const fullName = computed(
  () => `${firstName.value} ${lastName.value}`
)
```

COMPUTED PROPERTIES

Computed Properties

- Useful for operations too complex to sit in a template.
- Created by calling `computed()` with a getter function.

```
const message = ref('Hi There')
```

```
const reversedMessage = computed(() =>  
  message.value.split('').reverse().join('')  
)
```

```
// computed() returns a ref, so read it as reversedMessage.value  
// in script and as reversedMessage in the template.
```

Benefits of Computed Properties

- Unlike a plain function call, a computed property is cached and will not re-run unless a reactive value it read has changed.
- Can often be used instead of a watcher.
- A getter should have no side effects — do not mutate state or make requests inside one.

DEMO

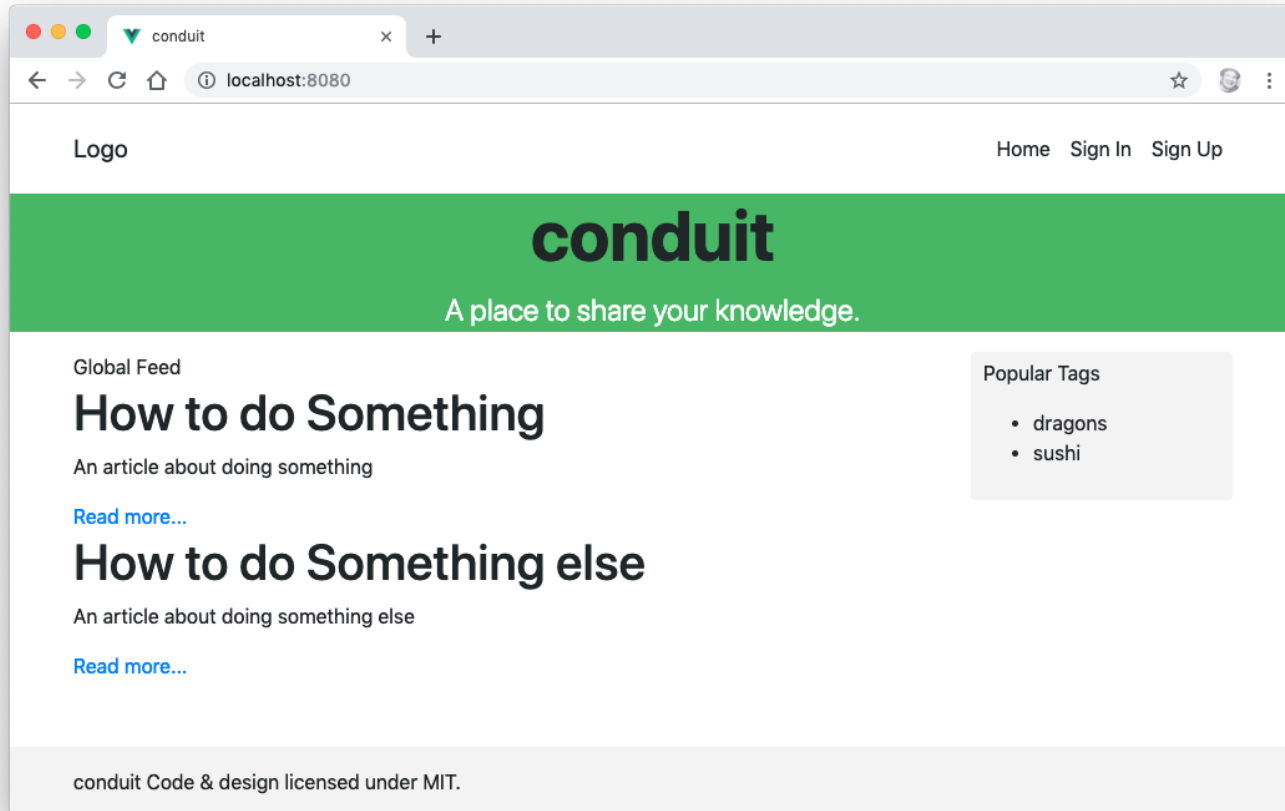
FILTERING / SORTING A LIST WITH A COMPUTED PROPERTY

Using Setters in Computed Properties

- Computed properties are getter-only by default.
- Pass an object with get and set when you need to write to one.

```
const fullName = computed({  
  get: () => `${firstName.value}  
${lastName.value}`,  
  set: (newValue) => {  
    const names = newValue.split(' ')  
    firstName.value = names[0]  
    lastName.value = names.at(-1)  
  }  
})
```

Lab 13: Computed Properties



```
import { reactive, ref } from 'vue'

const count = ref(0) // any value;
use .value
const user = reactive({ name: '' }) // objects
only; no .value

// A ref is the default choice. reactive() cannot
hold a
// primitive, and loses reactivity if you
destructure it.
```

REACTIVE STATE

ref and reactive

- State lives in refs, declared at the top of `<script setup>`.
- When a ref changes, every component that read it re-renders.
- Vue 3 tracks state with a Proxy, so properties added or deleted after creation are reactive too.
- `Vue.set` and `vm.$set` existed only to work around that limit in Vue 2, and were removed.

Vue.js Data Binding

- Use the v-bind directive (shorthand :) to bind an attribute to a reactive value — one-way binding.
- Vue keeps the attribute up to date with the value.
- Use the v-model directive to create two-way binding between an input and a ref.

v-bind directive

```
<!-- bind an attribute -->
```

```

```

```
<!-- dynamic attribute name -->
```

```
<button v-bind:[key]="value"></button>
```

```
<!-- with inline string concatenation -->
```

```

```

Value Binding

- Use v-bind to bind the value of a radio, checkbox, or select option to a dynamic value.

```
<input type="checkbox" v-model="toggle"  
true-value="yes" false-value="no">
```

```
<input type="radio" v-model="pick"  
:value="a">
```

2-WAY BINDING

v-model directive

- Use v-model to create a two-way data binding on form input, textarea, and select elements.
- v-model will ignore `value`, `checked` or `selected` attributes

Working with Arrays

- All array mutations are reactive, because Vue 3 observes the array through a Proxy:
 - push, pop, shift, unshift, splice, sort, reverse
 - items[0] = value — assignment by index
 - items.length = 0
- Vue 2 could not detect the last two, and needed Vue.set and splice as workarounds.
- Replacing the array wholesale is still often clearer:
 - items.value = items.value.filter(...)

Reactivity Caveats

- Proxy-based reactivity is deep and covers add and delete, but a few things still break tracking:
- Destructuring a reactive() object copies the values out
 - Use toRefs() — or a ref — instead
- Replacing a reactive() object wholesale drops the old proxy
- Reassigning a ref without .value inside <script setup>
- Map, Set, Date and class instances are supported, but a plain object is usually the simpler choice.

Destructuring and Reactivity

```
const state = reactive({ count: 0 })  
const { count } = state           // count is now a plain 0  
  
const { count } = toRefs(state)   // count is a ref – still  
live  
  
// The same rule applies to Pinia stores, which is why you  
// destructure them through storeToRefs().
```

v-bind shorthand

```
<!-- full syntax -->
```

```
<a v-bind:href="url"> ... </a>
```

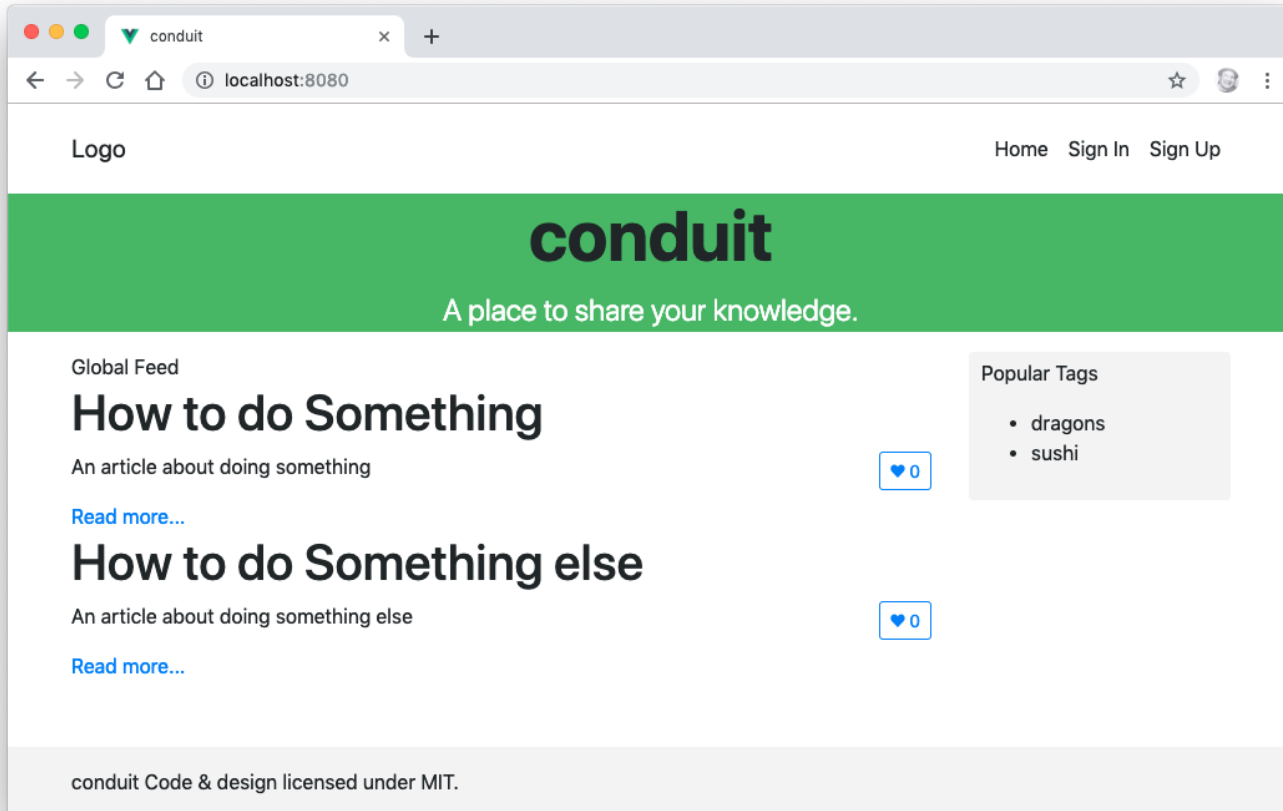
```
<!-- shorthand -->
```

```
<a :href="url"> ... </a>
```

```
<!-- shorthand with dynamic argument (2.6.0+) -->
```

```
<a :[key]="url"> ... </a>
```

Lab 14: Methods and State



Event Handling

- The v-on directive (shorthand @) creates an event listener.

```
<button @click="counter += 1">Add 1</button>
```

```
<p>The button has been clicked {{ counter }}  
times.</p>
```

```
import { ref } from 'vue'
```

```
const counter = ref(0)
```

Inline Event Handling

- Use small bits of JavaScript directly in the value of the v-on directive.

Event Handling with Methods

- Declare a function in <script setup> and call it from v-on.

```
const name = ref('Vue.js')
```

```
function greet(event) {  
  alert(`Hello ${name.value}!`)  
  if (event) alert(event.target.tagName)  
}
```

```
<button @click="greet">Greet</button>
```

Event Modifiers

- v-on has several modifiers that can be called to handle the details of DOM events.

`.stop`

`.prevent`

`.capture`

`.self`

`.once`

`.passive`

v-on shorthand

```
<!-- full syntax -->
```

```
<a v-on:click="doSomething"> ... </a>
```

```
<!-- shorthand -->
```

```
<a @click="doSomething"> ... </a>
```

```
<!-- shorthand with dynamic argument (2.6.0+) -->
```

```
<a @[event]="doSomething"> ... </a>
```


Listening to Child Component Events

- Child components declare and emit events

```
const emit = defineEmits(['enlarge-text'])  
<button @click="emit('enlarge-text')">Enlarge  
text</button>
```

- Parent components listen for them

```
<BlogPost @enlarge-text="postFontSize += 0.1"  
/>
```

```
// An undeclared listener falls through to the  
root element as a  
// native attribute, so always declare your  
events.
```

Emitting a Value With an Event

- emit can take further arguments

```
const emit = defineEmits(['enlarge-text'])  
<button @click="emit('enlarge-text', 0.1)">Enlarge  
text</button>
```

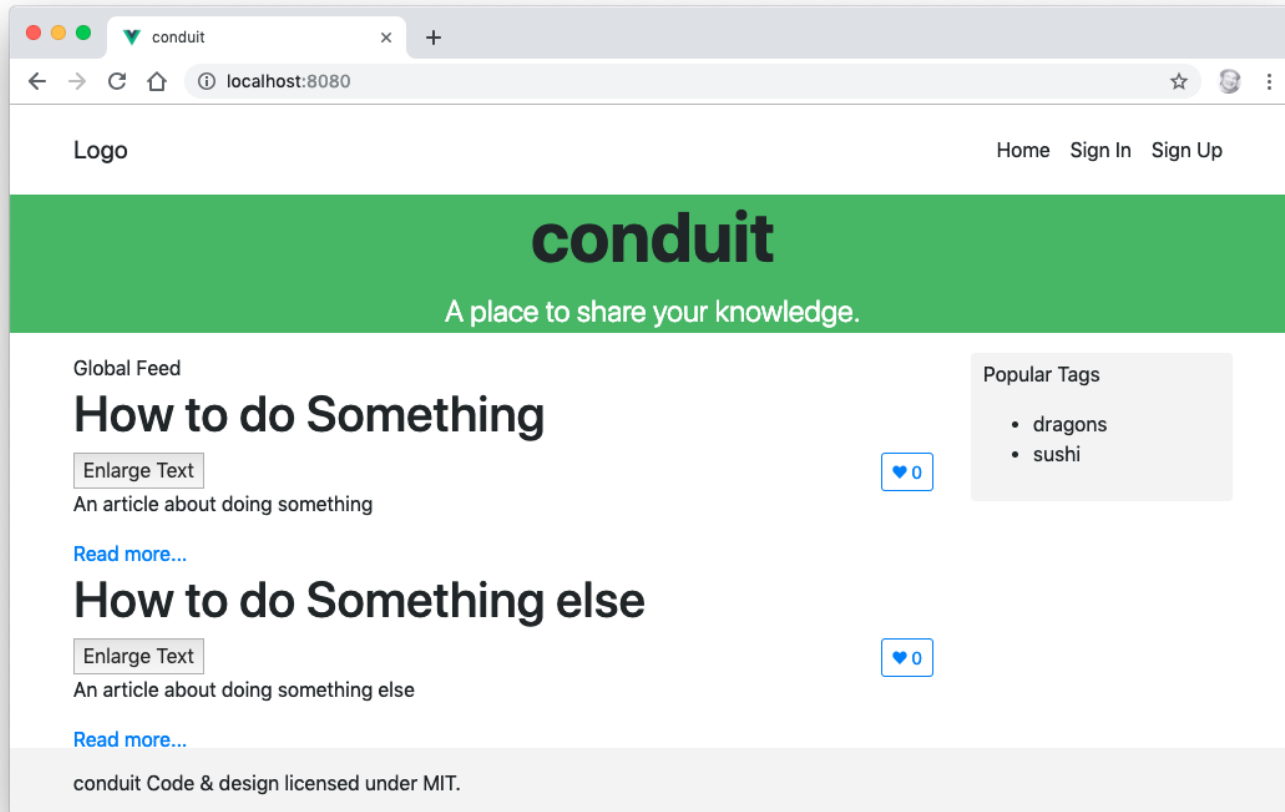
- The parent receives them as handler arguments

```
<BlogPost @enlarge-text="postFontSize += $event" />  
<BlogPost @enlarge-text="(n) => postFontSize += n"  
/>
```

// Vue 2 also offered \$on / \$off / \$once for an "event bus".

// Those were removed in Vue 3 – use a store or provide/inject.

Lab 15: Events



Watchers

- A general way to react to state changes.
- Most useful for asynchronous or expensive work in response to changing data.

```
const question = ref('')
const answer = ref('Ask me a question!')

watch(question, async (newQuestion,
oldQuestion) => {
  answer.value = 'Thinking...'
  answer.value = await getAnswer(newQuestion)
})
```

watch and watchEffect

`watch(source, callback)`

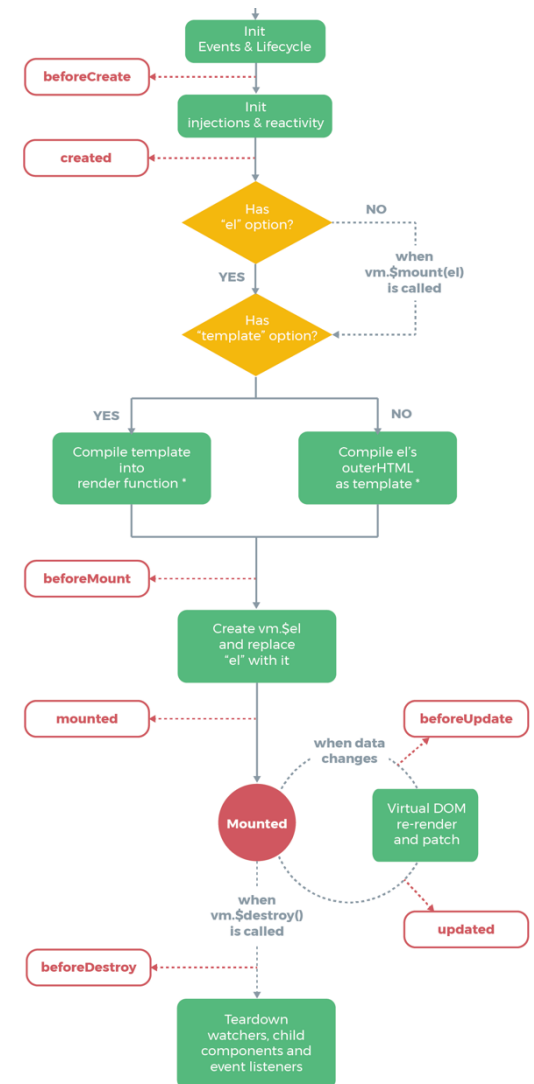
1. Explicit about what it tracks; gives you the old value
2. Lazy — the callback does not run until the source changes

`watchEffect(callback)`

1. Tracks whatever the callback reads, automatically
 2. Runs once immediately
1. Both return a stop function, and both stop automatically when the component unmounts.
 2. Reach for `computed()` first — a watcher is for side effects.

Component Lifecycle

- Functions imported from vue and called inside `<script setup>`
 - `onBeforeMount` / `onMounted`
 - `onBeforeUpdate` / `onUpdated`
 - `onBeforeUnmount` / `onUnmounted`
 - `onErrorCaptured`, `onActivated`, `onDeactivated`
- There is no `onCreated` — the body of `<script setup>` is that hook.
- Vue 2 named the last pair `beforeDestroy` and `destroyed`.



Setup (replacing beforeCreate / created)

- The body of `<script setup>` runs before the component is mounted.
- State and props exist; there is no DOM yet.
- This is where you declare refs, computed values and functions, and start any request whose result the first render does not need.

onBeforeMount

- Runs after the template has been compiled but before the first render.

```
onBeforeMount(() => {  
  console.log('about to render')  
})
```

- Rarely needed.

onMounted

- Full access to the rendered DOM through template refs.
- Use for measuring elements and integrating non-Vue libraries.

```
onMounted(async () => {  
  articles.value = await fetchArticles()  
})
```

- A child is mounted before its parent.

onBeforeUpdate

- Runs after state changes, before the DOM is patched.
- Use it to read the DOM as it was before the update — for example to record a scroll position.

onUpdated

- Runs after state changes and the DOM has re-rendered.
- Do not change state here without a guard — you will loop.
- To react to one specific value, prefer `watch()`.

onBeforeUnmount

- Fires right before teardown, while the component is still fully functional.
- Use it to clean up timers, subscriptions and listeners you created yourself.

onUnmounted

- Everything attached to the component has been torn down and removed from the DOM.
- Watchers and lifecycle hooks are stopped for you; anything you set up by hand is not.

Lifecycle Hooks in Practice

```
<script setup>
import { onMounted, onUnmounted, ref } from 'vue'

const seconds = ref(0)
• let timer

onMounted(() => {
  timer = setInterval(() => seconds.value++, 1000)
})

onUnmounted(() => clearInterval(timer))
</script>
```

Using Lifecycle Hooks

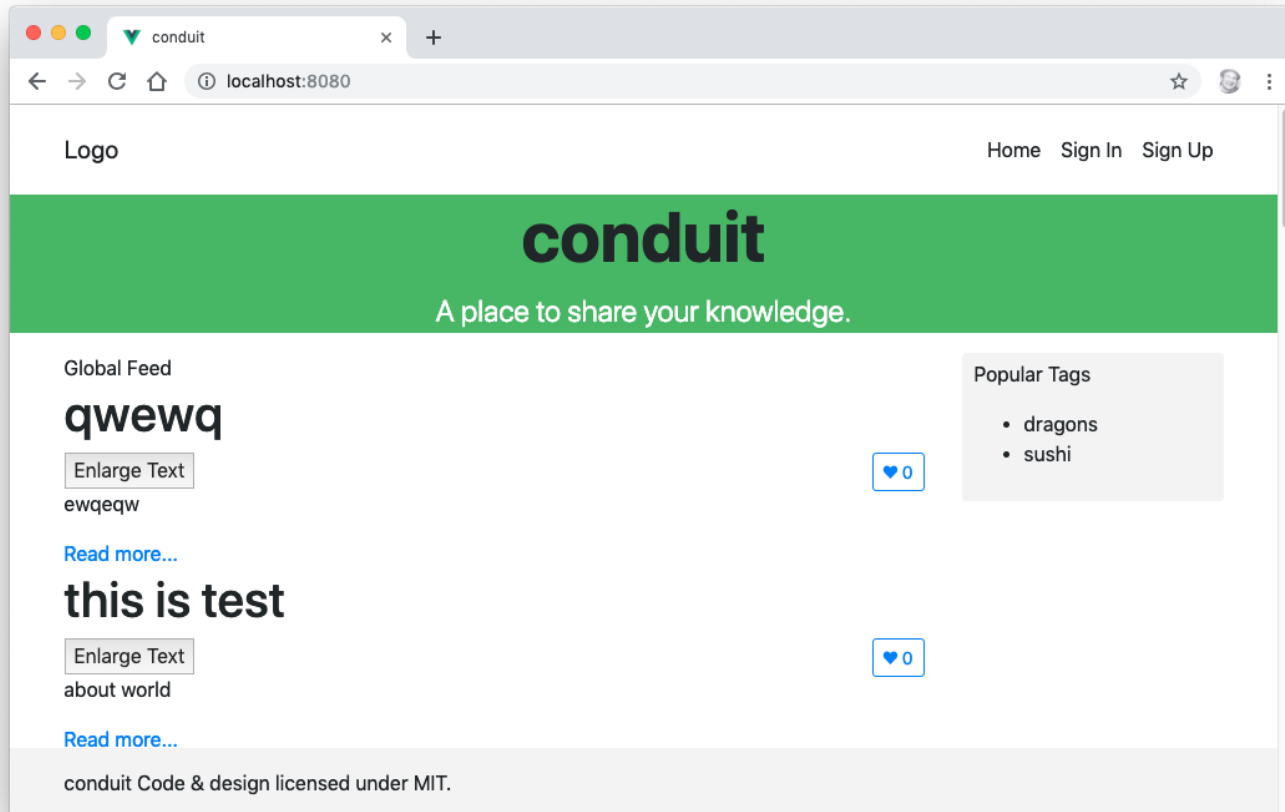
Hooks must be called synchronously in the body of `<script setup>`

Not inside a callback, a promise handler, or a condition

That is how Vue knows which component instance to attach them to

They can be called from a composable, because a composable is itself called synchronously from setup

Lab 16: Component Lifecycle



VUE AND FORMS

Input Bindings

```
<input v-model="yourAnswer">
```

```
<textarea v-model="yourAnswer"></textarea>
```

```
<input type="checkbox" id="yourAnswer" v-model="checked">
```

```
<input type="radio" id="first" value="First" v-model="picked">
```

```
<select v-model="selected">
```

```
<option>A</option>
```

```
<option>B</option>
```

```
<option>C</option>
```

```
</select>
```

2-way Binding Pros and Cons

- 2-way binding makes development easier
- 2-way binding makes testing more difficult
- v-model is sort of magic (which could be a pro or con)

```
<input v-model="searchText">
```

- does the same thing as:
- ```
<input
 :value="searchText"
 @input="searchText = $event.target.value" >
```

DEMO

# **CREATING A FORM WITH CHECKBOXES**

DEMO

# **CREATING A FORM WITH RADIO BUTTONS**

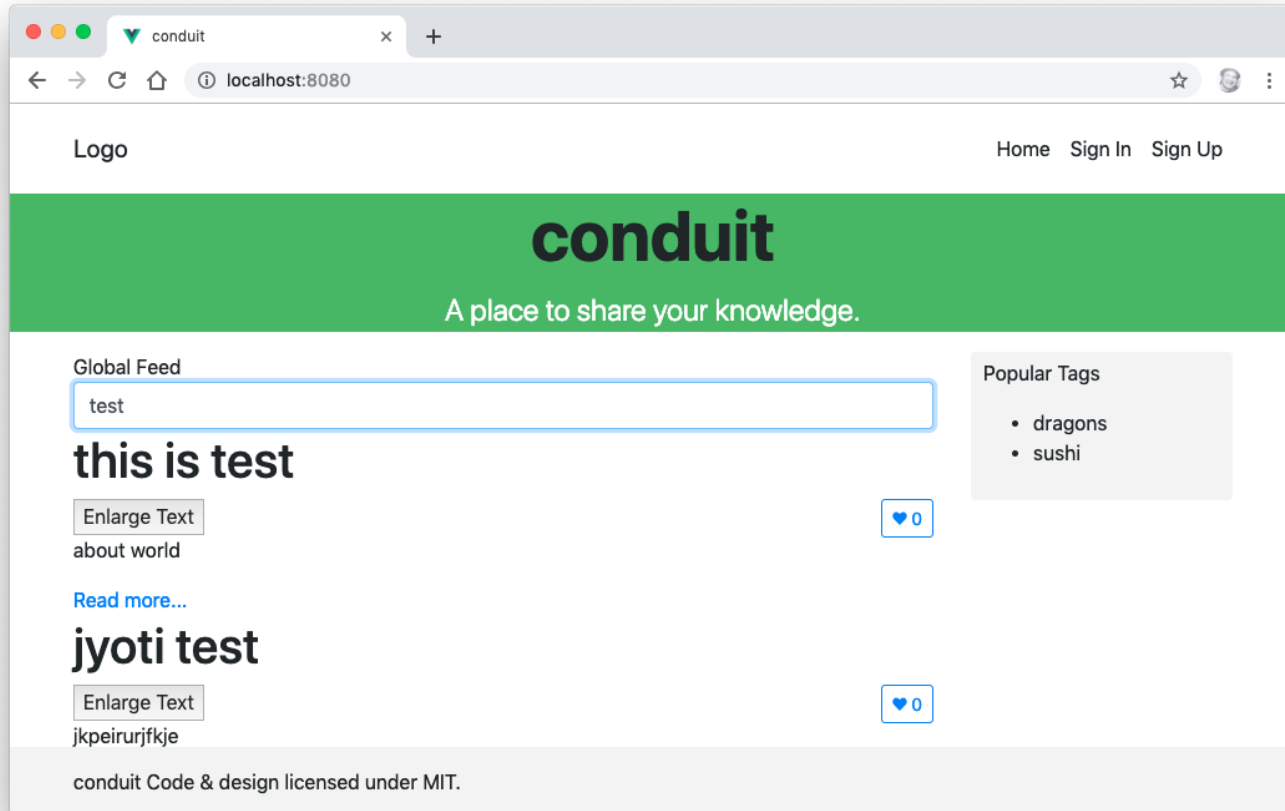
DEMO

# **CREATING A FORM WITH A SELECT ELEMENT**

DEMO

# INPUT MODIFIERS

# Lab 17: Forms





# **VUE COMPONENTS**

# What are Vue Components?

- A reusable piece of UI with its own state, template and styles.
- Written as a single-file component: .vue with `<script setup>`, `<template>` and `<style>`.
- State is declared with `ref()` and `reactive()`, so each instance automatically gets its own copy — this is what the `data()` function guaranteed in the Options API.
- A component may have more than one root element (a Fragment).

# Creating a Component

```
<script setup>
defineProps({
 todo: { type: Object, required: true }
})
</script>
```

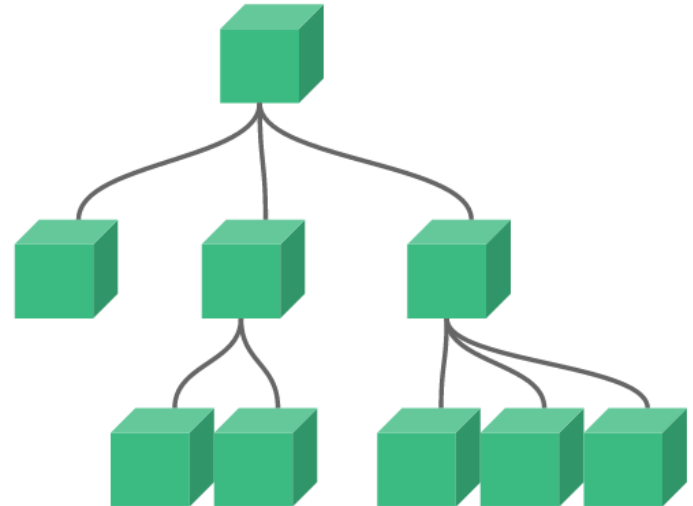
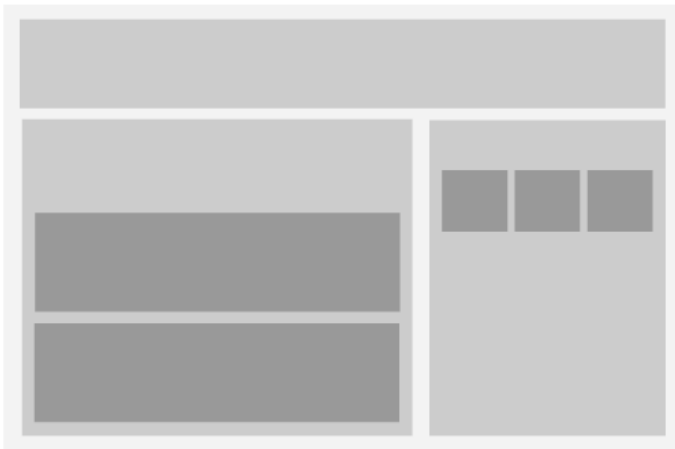
```
<template>
 {{ todo.text }}
</template>
```

// The file name is the component name: TodoItem.vue is used as

```
// <TodoItem />. Vue 2 wrote Vue.component("todo-item",
{ ... }).
```

# Organizing and Reusing Components

- Apps are usually organized as a tree of nested components.



# Naming Components

- Two options
  - PascalCase — the recommended default
    - Matches the file name, and the editor can resolve it
    - Distinguishes your components from native HTML elements
  - kebab-case
    - Required when writing templates directly in the DOM, because HTML tag names are case-insensitive
- A component imported in `<script setup>` can be used under the name you imported it as.

# Local Components

- The usual case: import the component where you use it.

```
<script setup>
```

- `import PopupWindow from './components/PopupWindow.vue'`

```
</script>
```

```
<template>
```

```
 <PopupWindow />
```

```
</template>
```

```
// The import is the registration. There is no
components option.
```

# Global Components

```
import { createApp } from 'vue'
```

- `import PopupWindow from './components/PopupWindow.vue'`
- `import App from './App.vue'`

```
const app = createApp(App)
```

```
app.component('PopupWindow', PopupWindow)
```

```
app.mount('#app')
```

```
// Registered on the app, not on a global Vue,
so a second app
```

```
// on the page does not see it.
```

# Global vs. Local

- Prefer local registration.
  - A reader can see where every component comes from
  - Unused components can be tree-shaken out of the bundle
  - The editor can jump to the definition
- Register globally only for components used on nearly every screen — an icon or a button primitive.



# Props

- Declare props with the `defineProps` compiler macro.

```
const props = defineProps({
 todo: { type: Object, required: true },
 size: { type: String, default: 'medium' }
})
```

```
// In the template write todo directly;
// in script go through props.todo.
// Props are read-only – emit an event to
ask for a change.
```

# Passing Data

- Pass data from parent to child components using props.
- Pass data to props using element attributes.

```
<todo-item todo = "wash the car" />
```

- A component can have as many props as you need
- Any value can be passed to any prop

# Using Props

```
<!-- parent -->
<TodoItem :todo="item" size="large" />
```

```
<!-- TodoItem.vue -->
<script setup>
defineProps(['todo', 'size'])
</script>
```

```
<template>
 {{ todo.text }}
</template>
```

# Fragments: Multiple Root Elements

- Vue 2 required a single root element. Vue 3 does not.

```
<template>
 <h1>{{ title }}</h1>
 <div v-html="content"></div>
</template>
```

- One caveat: with more than one root, Vue cannot decide where to put inherited attributes, so bind `$attrs` explicitly if the parent passes class or style.

# Using v-model on Components

- v-model works on components as it does on inputs

```
<CustomInput v-model="searchText" />
```

- does the same thing as
- ```
<CustomInput  
  :model-value="searchText"  
  @update:model-value="searchText = $event" />
```

```
// Vue 2 used the value prop and the input event. Vue 3  
renamed
```

```
// them to modelValue and update:modelValue, which frees up  
// value for ordinary use and allows several v-models per  
// component: v-model:title, v-model:body.
```

defineModel

- Since Vue 3.4 the whole prop-and-event pair has a shorthand.

```
<script setup>  
const model = defineModel()  
</script>
```

```
<template>  
  <input v-model="model">  
</template>
```

```
// model is a writable ref. Writing to it emits  
the update event  
// for you.
```

Content Distribution with Slots

- Slots allow you to pass the content of an element into a component.

```
<navigation-link url="/profile">  
  Your Profile  
</navigation-link>
```

- and then access that content inside the component

```
<a v-bind:href="url">  
  <slot></slot>  
</a>
```

Cool Things You Do With Slots

- Slots can contain any template code, including HTML and components.
- Slots can access the same scope as the rest of the template.
- Default slot content can be provided.

```
<slot>default content</slot>
```

- You can create named slots if you need to have multiple slots.

Naming Slots

- Parent component:

```
<base-layout>
  <template v-slot:header>
    Header content
  </template>
  <p>Main Content</p>
  <template v-slot:footer>
    Footer content
  </template>
</base-layout>
```

Using Named Slots

```
<div class="container">  
  <header>  
    <slot name="header"></slot>  
  </header>  
  <slot></slot>  
  <footer>  
    <slot name="footer"></slot>  
  </footer>  
</div>
```

DEMO

USING SLOTS

More fun with slots

- Scoped Slots
- Abbreviated Syntax for Default Slots
- Destructuring Slot Props
- Dynamic Slot Names
- Named Slots Shorthand
 - `v-slot:header` can be re-written as `#header`

```
<template #header>  
  <p>template content here</p>  
</template>
```

Lab 18: Slots

- Populate the list of tags in the 2nd column dynamically from the feed at /tags (using the same Base URL as the articles feed). Use Slots.
- List each article's tags underneath the article. Again, use slots.
- Make clicking on the tags in the tag box filter the list of articles.

Loading Components Asynchronously

- Wrap a dynamic import with `defineAsyncComponent`.

```
import { defineAsyncComponent } from 'vue'

const AsyncModal = defineAsyncComponent(() =>
  import('./components/Modal.vue')
)
```

- Vite gives the component its own chunk, downloaded on first use.
- Routes get this for free from a lazy route component.
- Pair it with `<Suspense>` to show a fallback while it loads.

COMPOSABLES

Composable Basics

- A composable is a function that uses the Composition API to encapsulate and reuse stateful logic.
- By convention it is named `use*` and lives in `src/composables`.
- It is called synchronously from `<script setup>`, so it can use lifecycle hooks and watchers on the caller's behalf.

Why Composables Replace Mixins

- A mixin merged its options into the component silently:
 - You could not tell where an inherited property came from
 - Two mixins declaring the same name collided, and the winner was decided by a merge strategy
 - A mixin could not take arguments
- A composable takes its inputs as parameters and returns its outputs, so both ends are visible at the call site.
- Mixins still work in Vue 3, but are documented as legacy.

Writing a Composable

```
import { computed, ref } from 'vue'

export function useArticleFilter(articles) {
  const searchDetails = ref('')

  const filterIt = computed(() =>
    articles.value.filter((a) =>
      a.title.includes(searchDetails.value)
    )
  )

  return { searchDetails, filterIt }
}
```

Using a Composable

```
<script setup>
import { ref } from 'vue'
import { useArticleFilter } from
'@/composables/useArticleFilter'

const articles = ref([])

const { searchDetails, filterIt } =
useArticleFilter(articles)
</script>

// Called twice, you get two independent filters –
something a
// mixin could never do.
```

Composable Conventions

Name it `use*` so readers know it may register hooks

Call it synchronously from `setup`, never in a callback

Accept refs or getters as arguments so callers can pass live state; `toValue()` normalises them

Return plain refs, so the caller can destructure freely

Clean up after yourself in `onUnmounted`

Community Composables

- VueUse — a large collection of ready-made composables
 - useLocalStorage, useFetch, useMediaQuery, useDebounce, ...
- Worth checking before writing your own
- vueuse.org

Lab 19: Composables

- Extract the `filterIt` computed property into a composable and use it from `ArticleList.vue`.

USING PINIA

WHY A STORE?

The Problem

- Props flow down; events flow up.
- That works until two distant components need the same state.
 - Passing it through every intermediate component ("prop drilling")
 - Two copies that drift out of sync
- A store gives that state one owner, outside the component tree.

Pinia

1. Vue's official state management library.
2. A store is a composable: you call `useSomeStore()` to get it.
3. state, getters and actions — no mutations, no dispatch, no commit.
4. Type-safe, and each store is its own module by construction.
5. Replaced Vuex, which is now in maintenance mode.

Pinia vs. Vuex

- Vuex Pinia
- state state
- getters getters
- mutations — (actions assign directly)
- actions + dispatch('name') actions, called as functions
- commit('NAME', payload) this.value = payload
- modules one file per store
- mapState / mapGetters storeToRefs

Installing Pinia

- `npm install pinia`

```
import { createPinia } from 'pinia'
```

```
const app = createApp(App)
```

```
app.use(createPinia())
```

```
app.mount('#app')
```

```
// Install Pinia before the router, so  
navigation guards can  
// reach the stores.
```

Defining a Store

```
import { defineStore } from 'pinia'

export const useCounterStore = defineStore('counter', {
  state: () => ({ count: 0 }),

  getters: {
    double: (state) => state.count * 2
  },

  actions: {
    increment() { this.count++ }
  }
})

// 'counter' is the store id, used by the DevTools.
```

Setup Stores

A store can also be written like <script setup>.

```
export const useCounterStore = defineStore('counter', () => {  
  const count = ref(0) // state  
  const double = computed(() => count.value * 2) // getter  
  
  function increment() { count.value++ } // action  
  
  return { count, double, increment }  
})  
  
// Same store, different syntax. Use whichever reads better;  
// setup stores compose more easily, option stores map more  
// directly onto a Vuex module you are porting.
```

State

- state is a function that returns a fresh object.
- Read and write it directly on the store:

```
const counter = useCounterStore()
```

- `counter.count++`
- `counter.$patch({ count: 10 })` // several changes at once
- `counter.$reset()` // back to the initial state

Getters

- Computed properties for a store.
 - Receive state as their first argument.
 - Cached, like any computed property.
 - `getters: {`
 `doneTodos: (state) => state.todos.filter((t)`
 `=> t.done),`
 `doneCount() { return this.doneTodos.length }`
 `}`
- `// Use `this` (and a normal function) to reach other getters.`

Using a Store in a Component

```
<script setup>
import { useCounterStore } from
  '@stores/counter'

const counter = useCounterStore()
</script>

<template>
  <p>{{ counter.count }} / {{ counter.double
  }}</p>
  <button
    @click="counter.increment()">+1</button>
</template>
```

storeToRefs

- A store is reactive, so destructuring it copies values out and breaks reactivity.

```
const { count, double } = counter           // WRONG – not  
reactive
```

```
import { storeToRefs } from 'pinia'
```

```
const { count, double } = storeToRefs(counter)  //  
refs
```

```
const { increment } = counter                 //  
actions are
```

```
plain functions                               //
```

```
// storeToRefs is the replacement for mapState and  
mapGetters.
```

Actions

- An action is just a method on the store.
- It may be async, and it assigns to state through this.

- `actions: {`
 `async fetchArticles(params) {`
 `this.isLoading = true`
 `try {`
 `const { data } = await`
`ArticlesService.query(params.type)`
 `this.articles = data.articles`
 `} finally {`
 `this.isLoading = false`
 `}`
 `}`
`}`

No Mutations

- Vuex separated synchronous mutations from asynchronous actions so the devtools could record every state change.
- Pinia tracks changes through the reactivity system instead, so the separation is unnecessary.
- The result: no mutation types, no commit, no string constants — one less indirection between the component and the state.
- `$subscribe` and `$onAction` are still there if you need to observe changes, for logging or persistence.

Multiple Stores

Every store is a separate module by construction.

src/stores/

auth.js -> useAuthStore

home.js -> useHomeStore

A store can use another store – just call its use-function.

// Vuex needed modules, namespacing, and 'module/action' strings.

Pinia and the DevTools

- Vue DevTools shows every registered store
 - Inspect and edit state live
 - A timeline of actions, with arguments and results
 - Time-travel debugging
- Stores appear as soon as they are first used.

Persisting Store State

- Pinia is extended with plugins.

```
pinia.use(({ store }) => {  
  store.$subscribe((_mutation, state) => {  
    localStorage.setItem(store.$id,  
      JSON.stringify(state))  
  })  
})  
  
// pinia-plugin-persistedstate packages this  
up.
```

Testing a Store

- A store is a plain function — no component needed.

```
beforeEach(() => setActivePinia(createPinia()))
```

```
it('increments', () => {  
  const counter = useCounterStore()
```

```
  counter.increment()
```

```
  expect(counter.count).toBe(1)  
})
```

```
// @pinia/testing adds createTestingPinia(), which  
stubs actions.
```


Lab 20: Implementing Pinia

SINGLE-PAGE APPLICATIONS

Creating an SPA with Vue Router

- With Vue Router you map components to routes and tell Vue Router where to render them.
- A route component is loaded lazily, so it gets its own bundle.

Vue Router: the Template

```
<script setup>
import { RouterLink, RouterView } from 'vue-router'
</script>

<template>
  <nav>
    <RouterLink :to="{ name: 'home'
}">Home</RouterLink>
    <RouterLink :to="{ name: 'login' }">Sign
in</RouterLink>
  </nav>

  <RouterView />
</template>
```

Vue Router: the JavaScript

```
import { createRouter,  
createWebHistory } from 'vue-  
router'
```

```
export default createRouter({  
  history:  
    createWebHistory(import.meta.  
env.BASE_URL),  
  routes: [  
    { name: 'home', path:  
      '/',  
      component: () =>  
import('@components/Home.vue  
) },  
    { name: 'login', path:  
      '/login',  
      component: () =>  
import('@components/Login.vu  
e') }
```

Lab 21: Routing

Lab 22: AJAX with Pinia

TESTING

Test-Driven Development

Objectives

- Learn the TDD steps

- Write assertions

- Create tests with Vitest

- Test components with Vue Test Utils

Goal of TDD

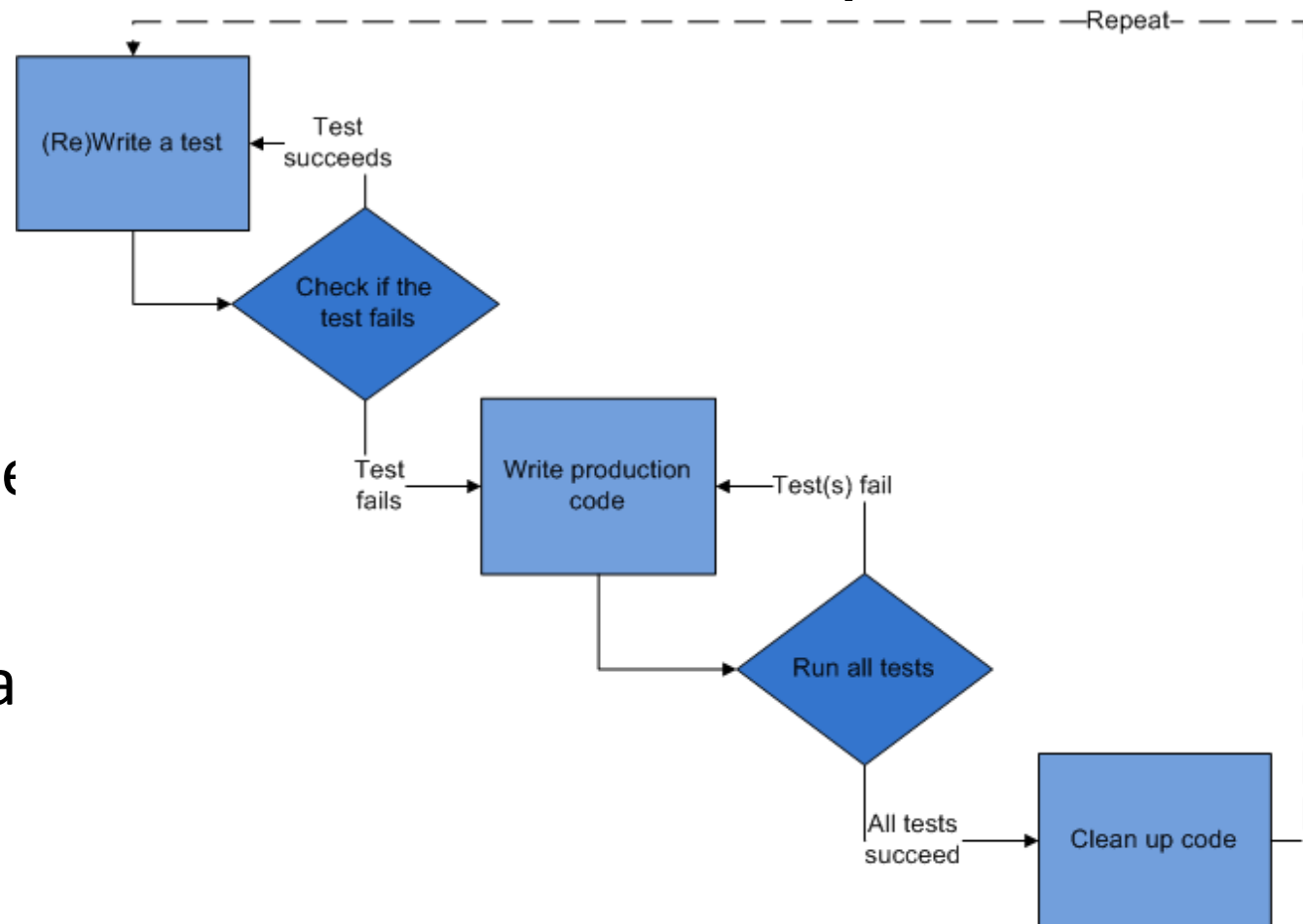
- Clean code that works.

The TDD Cycle

- Red
 - write a little test that doesn't work.
- Green
 - make the test work, as quickly as possible.
 - don't worry about doing it right.
- Refactor
 - eliminate duplication created in making the test work.

TDD Steps

- Write a test
- Check that test fails
- Write code
- Run test - pass
- Refactor
- Repeat



Red

- Write the story.
- Invent the interface you wish you had.
- Characteristics of a good tests:
 - Each test should be independent of the others.
 - Any behavior should be specified in only one test.
 - No unnecessary assertions
 - Test only one code unit at a time
 - Avoid unnecessary preconditions

Green

- Get the test to pass as quickly as possible.
- Three strategies:
 - Fake it
 - Do something, no matter how bad, to get the test to pass.
 - Use an obvious clean solution.
 - But don't try too hard!
 - Triangulation
 - only generalize code when you have two examples or more.
 - When the 2nd example demands a more general solution, then and only then do you generalize.

Refactor

- Make it right.
- Remove duplication.
- Improve the test.
- Repeat.
- Add ideas or things that aren't immediately needed to a todo list.

Assertions

- An expression that encapsulates testable logic
- Assertion styles

```
expect(buttonText).toBe('Go!');           // Vitest /  
Jest
```

```
expect(body).toBe.an('array');           // Chai
```

```
assert.deepEqual(obj1, obj2);           //  
node:assert
```

- Vitest's expect is Chai's, with a Jest-compatible surface.

JavaScript Testing Frameworks

- Vitest
 - Vite-native; shares your Vite config and transforms
- Jest
 - Long the default; needs its own transform setup for Vite projects
- Mocha + Chai
 - Runner and assertion library, chosen separately
- node:test
 - Built into Node.js since 18
- Playwright / Cypress
 - End-to-end, in a real browser

JS Exception Handling

```
function hello(name) {  
    return "Hello, " + name;  
}  
let result = hello("World");  
let expected = "Hello, World!";  
try {  
    if (result !== expected) throw new Error  
        ("Expected " + expected + " but got " +  
        result);  
} catch (err) {  
    console.log(err);  
}
```

Vitest Overview

Objectives

- Learn about the different rendering modes

- Learn about Vitest

- Write unit tests with Vitest

Vitest

- The test runner built for Vite
- Reuses your vite.config.js — aliases, plugins and .vue handling all work in tests without extra configuration
- Runs files named *.spec.js or *.test.js
- Simulates a browser environment with jsdom or happy-dom
- Jest-compatible API: describe, it, expect, vi.fn()
- Watch mode re-runs only the tests affected by your last save

How Vitest Works

- Suites describe your tests
- Specs contain expectations

```
import { describe, expect, it } from 'vitest'

describe('A suite is just a function', () => {
  it('and so is a spec', () => {
    expect(true).toBe(true)
  })
})

// Set globals: true in the config to skip the
import.
```

Test Suites

- Created using the describe function
- Contain one or more specs
- 2 params
 - Text description
 - Function

```
describe('Hello', () => {  
  // ...  
})
```

Specs

- Created using the it or test function
 - they're the same thing
- Contains one or more expectations
 - expectations === assertions

```
describe('hello', () => {  
  it('concat Hello and a name', () => {  
    expect(hello('World')).toBe('Hello,  
World!')  
  })  
})
```

Expectations

- AKA assertions
- Made using `expect` function,
 - Takes a value
- Chained to a Matcher
 - Takes the expected value

```
expect(actual).toEqual(expected);
```


Matchers

```
expect(fn).toThrow(e);  
expect(value).toBe(other); // Object.is  
expect(value).toEqual(other); // deep  
equality  
expect(value).toBeDefined();  
expect(value).toBeTruthy() / .toBeFalsy();  
expect(number).toBeGreaterThan(n) /  
.toBeLessThan(n);  
expect(value).toBeNull() / .toBeUndefined();  
expect(array).toContain(member);  
expect(string).toMatch(pattern);  
await expect(promise).resolves.toBe(value);
```

TDD vs. BDD

Test-Driven Development

- Focused on being useful for programmers

Behavior-Driven Development

- Focused on documentation for non-programmers
 - Features, not results
 - More verbose

TDD Example

```
describe('Counter', () => {  
  test('tick increases count to 1', () => {  
    const counter = new Counter()  
  
    counter.tick()  
  
    expect(counter.count).toBe(1)  
  })  
})
```

BDD Example

```
describe('Counter', () => {  
  it('should increase count by 1 after calling  
tick', () => {  
    const counter = new Counter()  
    const expected = counter.count + 1  
  
    counter.tick()  
  
    expect(counter.count).toBe(expected)  
  })  
})
```

Mocking

- `vi.fn()` — a mock function that records how it was called
- `vi.mock()` — replace a whole module
- `vi.spyOn()` — wrap an existing method, keeping the original
- `vi.useFakeTimers()` — control `setTimeout` and `setInterval`
- `vi.stubGlobal()` — replace a global such as `fetch`

Mock Function

```
import { expect, it, vi } from 'vitest'

const mockCallback = vi.fn()

[0, 1].forEach(mockCallback)

// The mock function is called twice
expect(mockCallback).toHaveBeenCalledTimes(2)

// It was called with 0, and then with 1
expect(mockCallback).toHaveBeenNthCalledWith(1, 0,
0, [0, 1])
```

Mocking a Module

- Replace a module your component imports, so the test never hits the network.

```
vi.mock('@common/api.service', () => ({  
  default: {  
    query: vi.fn().mockResolvedValue({ data: {  
articles: [] } })  
  }  
}))
```

```
// vi.mock is hoisted above the imports, so it  
applies even  
// though it is written after them.
```

Resetting Mocks

- Mocks remember every call, so state leaks between tests.

```
vi.clearAllMocks()    // forget recorded  
calls
```

```
vi.resetAllMocks()    // also drop the  
implementation
```

```
vi.restoreAllMocks()  // undo every spyOn
```

```
// Or set restoreMocks: true in the Vitest  
config.
```


Fake Timers

```
vi.useFakeTimers()
```

```
const spy = vi.fn()  
setTimeout(spy, 5000)
```

```
vi.advanceTimersByTime(5000)
```

```
expect(spy).toHaveBeenCalled()
```

```
vi.useRealTimers()
```

Snapshot Testing

1. Renders a component
2. Writes a snapshot file on the first run
3. Compares later runs against it, and fails if the output differs
4. Update deliberately with vitest -u, and review the diff
5. Best for output that should rarely change; a poor substitute for an assertion about behaviour

Sample Snapshot Test

```
it('should render content correctly', () => {  
  const wrapper = mount(Component, {  
    global: { plugins: [pinia, router] }  
  })  
  
  expect(wrapper.html()).toMatchSnapshot()  
})
```

Vue Test Utils

- The official unit-testing utility library for Vue.
- Version 2 is the one built for Vue 3.
- Changes from version 1:
- `propsData` is now `props`
- `createLocalVue` is gone – use `global.plugins`
- `stubs`, `mocks` and `provide` moved under `global`
- `setValue` and `trigger` return a promise; await them

mount

- Creates a wrapper containing the mounted and rendered component, including all of its children.

```
const wrapper = mount(Header, {  
  props: { title: 'Conduit' },  
  global: { plugins: [router] }  
})
```

shallowMount

- Creates a wrapper containing the mounted and rendered component, with its child components stubbed out.
- Useful when a child would fetch data or need its own plugins.

```
const wrapper = shallowMount(Home, {  
  global: { plugins: [pinia, router] }  
})
```

Lab 23: Testing with Vitest

TESTING IN A REAL BROWSER

Browser and End-to-End Testing

- jsdom is a fast approximation of a browser, not a browser.
- Vitest browser mode runs the same tests in a real one, driven by Playwright or WebdriverIO.
- Playwright and Cypress drive the whole application end to end.
- Karma, which earlier versions of this course covered, was deprecated in 2023.

TRANSITIONS AND ANIMATIONS

<Transition>

- A built-in wrapper component
- Creates enter/leave transitions for the element inside it
- Can be used with:
 - conditional rendering (v-if)
 - conditional display (v-show)
 - dynamic components
 - a component root node
- <TransitionGroup> does the same for a list rendered with v-for

How <Transition> works

- When the wrapped element is inserted or removed...

```
<button @click="show =  
!show">Toggle</button>
```

```
<Transition name="fade">  
  <p v-if="show">hello</p>  
</Transition>
```

The transition classes

- Vue looks for CSS transitions or animations named after the transition's name attribute, and applies these classes in order:
 - v-enter-from — the starting state, applied before insertion
 - v-enter-active — applied for the whole entering phase
 - v-enter-to — the end state of the enter transition
 - v-leave-from — the starting state of the leave transition
 - v-leave-active — applied for the whole leaving phase
 - v-leave-to — the end state, applied until the animation finishes
- Vue 2 called the first and fourth v-enter and v-leave. Renaming them is the most common reason an old example stops animating.

Defining the CSS

- .fade-enter-active,
- .fade-leave-active {
 transition: all 0.3s ease;
}

- .fade-enter-from,
- .fade-leave-to {
 transform: translateX(10px);
 opacity: 0;
}

JavaScript hooks

- **<Transition**

```
@before-enter="beforeEnter"
```

```
@enter="enter"
```

```
@after-enter="afterEnter"
```

```
@enter-cancelled="enterCancelled"
```

```
@before-leave="beforeLeave"
```

```
@leave="leave"
```

```
@after-leave="afterLeave"
```

```
@leave-cancelled="leaveCancelled"
```

```
:css="false"
```

- **>**

```
<p v-if="show">hello</p>
```

```
</Transition>
```

Using the JavaScript hooks

- Declare the handlers in <script setup>.

```
function beforeEnter(el) {  
  el.style.opacity = 0  
}
```

```
function enter(el, done) {  
  // call done() when the animation finishes  
  done()  
}
```

```
// Set :css="false" so Vue does not also wait  
for CSS.
```


Custom Transition Classes

- Override the generated class names to plug in a CSS animation library:
 - enter-from-class
 - enter-active-class
 - enter-to-class
 - leave-from-class
 - leave-active-class
 - leave-to-class
- Vue 2 named the first and fourth enter-class and leave-class.

Custom Transition Classes Example

```
<link
```

```
  href="https://cdn.jsdelivr.net/npm/animate.css@4.1.1"
  rel="stylesheet">
```

```
<Transition
```

```
  enter-active-class="animate__animated
  animate__tada"
```

```
  leave-active-class="animate__animated
  animate__bounceOutRight"
```

```
>
```

```
  <p v-if="show">hello</p>
```

```
</Transition>
```

Lab 24: Transitions and Animation

- Add a Remove From Feed icon (maybe a trash can) to each article.
- When clicked, it should remove the article from the feed.
- Animate the transition of the article from showing to hiding

ES2015 (ES6) and Beyond

Objectives

- Learn what's new
- Use arrow functions and block-scoped variables
- Create generator functions
- Use classes and modules
- Transpile ES6 code to ES5 with Babel

Variable Scoping with `const` and `let`

- `const` creates constants
 - "immutable variables"
 - Cannot be reassigned new content
 - The assigned content isn't immutable, however,
 - If you assign an object to a constant, the object can still be changed.
- `let` creates block-scoped variables
 - Main difference between `let` and `var` is that the scope of `var` is the entire enclosing function.
 - Redeclaring a variable with `let` raises a syntax error
 - No hoisting
 - Referencing a variable in the block before the declaration results in a `ReferenceError`.

let vs. var

var

```
var a = 5;  
var b = 10;
```

```
if (a===5) {  
    var a = 4;  
    var b = 1;  
}
```

```
console.log(a); // 4  
console.log(b); // 1
```

let

```
let a = 5;  
let b = 10;
```

```
if (a===5) {  
    let a = 4;  
    let b = 1;  
}
```

```
console.log(a); // 5  
console.log(b); // 10
```

Block-scoped Functions

ES5

```
(function () {  
  var foo = function () {  
    return 1;  
  }  
  console.log(foo()); // 1  
  
  (function () {  
    var foo = function() {  
      return 2;  
    }  
    console.log(foo()); // 2  
  })();  
  
  console.log(foo()); // 1  
})();
```

ES6

```
{  
  function foo () { return 1; }  
  console.log(foo()); // 1  
  {  
    function foo () {  
      return 2;  
    }  
    console.log(foo()); // 2  
  }  
  console.log(foo()); // 1  
}
```

Arrow Functions

- aka "fat arrow" functions
- A more concise syntax for writing functions
 - ES5 (old way)

```
function increment(v) {  
  return v+1;  
}
```

- Arrow function

```
increment = (v) => {v+1};
```


Arrow Function Parameters

- Surround parameter names with parentheses

`(param1,param2,param3) => { statements }`

- Parentheses are optional when there's only one parameter name

`singleParam => { statements }`

Arrow Functions (cont.)

- More intuitive handling of current object context.

- ES6

```
this.nums.forEach((v) => {  
  if (v % 5 === 0)  
    this.fives.push(v);  
});
```

- ES5

```
var self = this;  
this.nums.forEach(function (v) {  
  if (v % 5 === 0)  
    self.fives.push(v);  
});
```

Default Parameter Handling

- ES6

```
function myFunc (x, y = 0, z = 13) {  
    return x + y + z;  
}
```

- ES5

```
function f (x, y, z) {  
    if (y === undefined)  
        y = 0;  
    if (z === undefined)  
        z = 13;  
    return x + y + z;  
};
```

Rest Parameter

- Aggregation of remaining arguments into single parameter of variadic functions.

```
function myFunc (x, y, ...a) {  
    return (x + y) * a.length;  
}  
console.log(myFunc(1, 2, "hello", true, 7));
```

<http://jsbin.com/pisupa/edit?js,console>

Spread Operator

- Spreading of elements of an iterable collection (like an array or a string) into both literal elements and individual function parameters.

```
var params = [ "hello", true, 7 ];  
var other = [ 1, 2, ...params ];  
console.log(other); // [1, 2, "hello", true, 7]
```

```
console.log(MyFunc(1, 2, ...params));
```

```
var str = "foo";  
var chars = [ ...str ]; // [ "f", "o", "o" ]
```

<http://jsbin.com/guxika/edit?js,console>

Template Literals

- String Interpolation

```
var customer = { name: "Penny" }  
var order = { price: 4, product: "parts", quantity: 6 }  
message = `Hi, ${customer.name}. Thank you for your order  
of ${order.quantity} ${order.product} at ${order.price}.`;
```

<http://jsbin.com/pusako/edit?js,console>

Template Literals (cont)

- Raw String Access

Allows you to access the raw template string content (without interpreting backslashes)

```
function tag(strings, ...values) {  
    console.log(strings.raw[0]);  
    // "string text line 1 \\n string text line 2"  
}  
tag`string text line 1 \\n string text line 2`;
```

- <http://jsbin.com/donibif/edit?js,console>

Enhanced Object Properties

- Property Shorthand
 - Shorter syntax for properties with the same name and value

- ES5

```
obj = { x: x, y: y };
```

- ES6

```
obj = { x, y };
```


Enhanced Object Properties

- Computed names in object property definitions

```
let obj = {  
  customer: "Nigel",  
  [ "order" + getOrderNum() ]: 10  
};
```

<http://jsbin.com/wejuqe/edit?js,console>

Method notation in object property definitions

ES6

```
obj = {  
  foo (a, b) {  
  },  
  bar (x, y) {  
  }  
};
```

ES5

```
obj = {  
  foo: function(a, b) {  
  },  
  bar: function(x, y) {  
  }  
};
```

Array Matching

- Intuitive and flexible destructuring of Arrays into individual variables during assignment

```
var list = [ 1, 2, 3 ];  
var [ a, , b ] = list; // a = 1 , b = 3  
[ b, a ] = [ a, b ];
```

<http://jsbin.com/yafage/edit?js,console>

Object Matching

- Flexible destructuring of Objects into individual variables during assignment

```
var { a, b, c } = { a:1, b:2, c:3 };  
console.log(a); // 1  
console.log(b); // 2  
console.log(c); // 3
```

<http://jsbin.com/kuvizu/edit?js,console>

Symbol Primitive

- A **unique** and **immutable** data type.
- May be used as an identifier for object properties.
- Examples:

```
var sym1 = Symbol();  
var sym2 = Symbol("foo");  
var sym3 = Symbol("foo"); // Symbol("foo") !==  
Symbol("foo")
```

- Well-known Symbols
 - Built-in symbols, for example `Symbol.iterator`, which returns the default iterator for an object.

User-defined Iterators

- Customizable iteration behavior for objects
- In order to be iterable, an object must implement the iterator method -- the object, or one of the objects up its prototype chain) must have a property with a `Symbol.iterator` key.

```
var myIterable = {}  
myIterable[Symbol.iterator] = function* () {  
    yield 1;  
    yield 2;  
    yield 3;  
};  
[...myIterable] // [1, 2, 3]
```

For-Of Operator

- Convenient way to iterate over all values of an iterable object

```
let fibonacci = {
  [Symbol.iterator]() {
    let pre = 0, cur = 1
    return {
      next() {
        [pre, cur] = [cur, pre + cur]
        return { done: false, value: cur }
      }
    }
  }
}

for (let n of fibonacci) {
  if (n > 1000)
    break;
  console.log(n);
}
```

<http://jsbin.com/nururoz/edit?js,console>

Creating and Consuming Generator Functions

- A generator is a special type of function that works as a factory for iterators.

```
function* idMaker() {  
    var index = 0;  
    while(true)  
        yield index++;  
}  
var gen = idMaker();  
console.log(gen.next().value); // 0  
console.log(gen.next().value); // 1
```

- <http://jsbin.com/pozite/edit?js,console>

Class Definition

- ES6 introduces more OOP-style classes
- Can be created with Class declaration or Class expression

Class Declaration

```
class Square {  
    constructor (height, width) {  
        this.height = height;  
        this.width = width;  
    }  
}
```

Class Expressions

- Can be unnamed

```
var Square = class {  
    constructor(height,width) {  
        this.height = height;  
        this.width = width;  
    }  
};
```

- Or named

```
class Square {  
    constructor(height, width) {  
        this.height = height;  
        this.width = width;  
    }  
};
```

Class Inheritance

```
class Rectangle extends Shape {  
    constructor (id, x, y, width, height) {  
        super (id, x, y);  
        this.width = width;  
        this.height = height;  
    }  
}  
  
class Circle extends Shape {  
    constructor (id, x, y, radius) {  
        super (id, x, y);  
        this.radius = radius;  
    }  
}
```

Beyond ES2015

- ES2016
 - aka ES7
 - New Features
 - Exponentiation operator (**)
 - Array.prototype.includes
- ES2017
 - aka ES8
 - New Features
 - async/await
- ES.Next
 - Dynamic name for upcoming version

this, promises, map

ADVANCED JAVASCRIPT TOPICS

"use strict"

- Introduced with ES5
- Tells browsers to check for common errors and bad practices at run time
- Opts in to restricted variant of JavaScript
- To invoke:
 - put `"use strict;"` before any other statements

- Can also be invoked for functions:

```
function myStrictFunction() {  
    'use strict';  
    ...  
}
```

- Is implied in ES6 Modules and Classes
- Is redundant in properly written ES6 code.

Understanding `this`

- Allows functions to be reused with different context.
- Which object should be focal when invoking a function.

4 Rules of `this`

- Implicit Binding
- Explicit Binding
- New Binding
- Window Binding

What is `this`?

- When was function invoked?
- We don't know what `this` is until the function is invoked.

Implicit Binding

- `this` refers to the object to the left of the dot.

```
var author = {  
  name: 'Chris',  
  homeTown: 'Detroit',  
  logName: function() {  
    console.log(this.name);  
  }  
});
```

```
author.logName();
```

Explicit Binding

- `.call`, `.apply`, `.bind`

```
var logName = function() {  
    console.log(this.name);  
}
```

```
var author = {  
    name: 'Chris',  
    homeTown: 'Detroit'  
}  
logName.call(author);
```

Explicit Binding with .call

- Calls a function with a given `this` value and the arguments given individually.

```
var logName = function(lang1) {  
    console.log(this.fname + this.lname + lang1);  
};  
  
var author = {  
    fname: "Chris",  
    lname: "Minnick"  
};  
  
var language = ["JavaScript", "HTML", "CSS"];  
logName.call(author, language[0]);
```

Explicit binding with .apply

- Calls a function with a given `this` value and the arguments given as an array

```
logName = function(food1, food2, food3) {  
    console.log(this.fname + this.lname);  
    console.log(food1, food2,  
        food3);  
};  
  
var author = {  
    fname: "Chris",  
    lname: "Minnick"  
};  
  
var favoriteFoods= ['Tacos', 'Soup', 'Sushi'];  
  
logName.apply(author, favoriteFoods);
```

Explicit Binding with .bind

- Works the same as .call, but returns `new` function rather than immediately invoking the function

```
logName = function(food) {  
    console.log(this.fname + " " + this.lname +  
        "\'s Favorite Food was " + food);  
};  
  
var person = {  
    fname: "George",  
    lname: "Washington"  
};  
  
var logMe = logName.bind(person, "Tacos");  
  
logMe();
```

<http://jsbin.com/xikuzog/edit?js,console>

new Binding

- When a function is invoked with the `new` keyword, then this keyword inside the object is bound to the new object.

```
var City = function (lat,long,state,pop) {  
  this.lat = lat;  
  this.long = long;  
  this.state = state;  
  this.pop = pop;  
};  
var sacramento = new City(38.58,121.49,"CA",480000);  
console.log (sacramento.state);
```


window Binding

- What happens when no object is specified or implied
- `this` defaults to the `window` object

```
var logName = function() {  
    console.log(this.name);  
}  
var author = {  
    name: 'Chris',  
    homeTown: 'Detroit'  
}  
logName(author); //undefined(error in 'strict' mode)  
window.author = "Harry";  
logName(author); // "Harry"
```

Array.map()

Array.map()

- Creates a new array with the results of calling a provided function on every element in this array.

- **Syntax**

```
var new_array = arr.map(callback)
```

- **Parameters passed to the callback**

currentValue

- The current element being processed

index

- The index (number) of the current element

array

- The array map was called upon

PROMISES

What Are Promises?

- An abstraction for asynchronous programming
- Alternative to callbacks
- A promise represents the result of an async operation
- Is in one of three states
 - pending – the initial state of a promise
 - fulfilled – represents a successful operation
 - rejected – represents a failed operation

Promises vs. Event Listeners

- Event listeners are useful for things that can happen multiple times to a single object.
- A promise can only succeed or fail once.
- If a promise has succeeded or failed, you can react to it at any time.

```
readJSON(filename).then(success, failure);
```

Why Use Promises?

- Chain them together to transform values or run additional async actions
- Cleaner code
 - Avoid problems associated with multiple callbacks
 - Callback Hell
 - Christmas Tree
 - Tower of Babel
 - Etc

Demo: Callback vs. Promise

- **Callback**

```
fs.readFile('text.txt', function(err, file) {  
    if (err) {  
        //handle error  
    } else {  
        console.log(file.toString());  
    }  
});
```

- **Promise**

```
readText('text.txt')  
    .then(function(data) {  
        console.log(data.toString());  
    }, console.error)  
);
```

Using Promises

```
const fs = require('fs');

function readFileAsync (file, encoding) {
  return new Promise(function (resolve, reject) {
    fs.readFile(file, encoding, function (err, data) {
      if (err) return reject(err);
      resolve(data);
    })
  })
}

readFileAsync('myfile.txt')
  .then(console.log, console.error);
```


Async / Await

Simplifies using promises.

```
async function f() {  
    return 1;  
}
```

An async function always returns a promise.

```
f().then(alert);
```

You can use the await keyword inside an async function to wait until the promise resolves.

```
async function getUser() {  
    let response = await fetch('http://someurl.com/user.json');  
    let user = await response.json();  
    return user;  
}
```

Disclaimer and Copyright

Disclaimer

- WatzThis? takes care to ensure the accuracy and quality of this courseware and related courseware files. We cannot guarantee the accuracy of these materials. The courseware and related files are provided without any warranty whatsoever, including but not limited to implied warranties of merchantability or fitness for a particular purpose. Use of screenshots, product names and icons in the courseware are for editorial purposes only. No such use should be construed to imply sponsorship or endorsement of the courseware, nor any affiliation of such entity with WatzThis?.

Third-Party Information

- This courseware contains links to third-party web sites that are not under our control and we are not responsible for the content of any linked sites. If you access a third-party web site mentioned in this courseware, then you do so at your own risk. We provide these links only as a convenience, and the inclusion of the link does not imply that we endorse or accept responsibility for the content on those third-party web sites. Information in this courseware may change without notice and does not represent a commitment on the part of the authors and publishers.

Copyright

- Copyright 2026, WatzThis?. All Rights reserved. Screenshots used for illustrative purposes are the property of the software proprietor. This publication, or any part thereof, may not be reproduced or transmitted in any form by any means, electronic or mechanical, including photocopying, recording, storage in an information retrieval system, or otherwise, without express written permission of WatzThis, 567 Commercial St, Astoria, OR 97103. www.watzthis.com

Help us improve our courseware

- Please send your comments and suggestions via email to info@watzthis.com